NAME: T.ABIRAMI

VTU:28920

COURSE CODE:10211CS225

COURSE NAME:PROBLEM SOLVING AND TESTING USING JAVA

SLOT:S6-2/L1L14

FACULTY NAME:DR.K.BOOPALAN

SUBMISSION DATE:09/10/2026

INDEX:

S.NO	TITLE	PAGE.NO
1	Lab tasks(1-10)	3-19
2	Week 1	20-37
3	Week 2	38-57
4	Week 3	58-71
5	Week 4	72-86
6	Week 5	87-102
7	Week 6	103-120
8	Week 7	121-139
9	Week 8	140-161
10	Week 9	162-174
11	Vac certificate	175

PSTJ LAB TASKS

EXPERIMENT 1: Salary Transformation System Using Functional Programming Techniques

Java Program:

```
import java.util.*;
import java.util.stream.Collectors;
public class SalaryTransformationSystem {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 // Read number of employees
 int n = sc.nextInt();
 List<Integer> salaries = new ArrayList<>();
 // Read salaries
 for (int i = 0; i < n; i++) {
 salaries.add(sc.nextInt());
 }
 // Increase salary by 10%
 List<Integer> updatedSalaries = salaries.stream()
 .map(salary -> (int) (salary * 1.10))
 .collect(Collectors.toList());
 // Print updated salaries
 for (int salary : updatedSalaries) {
 System.out.print(salary + " ");
}
sc.close();
 }
}
```

Sample Input:

5

1000 2000 3000 4000 5000

Sample Output:

1100 2200 3300 4400 5500

EXPERIMENT 2: Design and Implementation of a Real-Time Stream Analytics Engine Using Stream Processing Concepts

Java Program:

```
import java.util.*;
import java.util.stream.*;
class SensorReading {
 String sensorId;
 double temperature;
 SensorReading(String sensorId, double temperature) {
 this.sensorId = sensorId;
 this.temperature = temperature;
 }
 public String getSensorId() {
 return sensorId;
 }
 public double getTemperature() {
 return temperature;
 }
}
public class RealTimeStreamAnalyticsEngine {
 public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
```

```
List<SensorReading> readings = new ArrayList<>();
for (int i = 0; i < n; i++) {
String id = sc.next();
double temp = sc.nextDouble();
readings.add(new SensorReading(id, temp));
}
Map<String, Double> averageMap = readings.stream()
.filter(r -> r.getTemperature() > 50)
.collect(Collectors.groupingBy(
SensorReading::getSensorId,
Collectors.averagingDouble(SensorReading::getTemperature)
));
averageMap.entrySet().stream()
.sorted(Map.Entry.<String, Double>comparingByValue().reversed())
.forEach(entry ->
System.out.println(entry.getKey() + " " +
String.format("%.1f", entry.getValue())));
}
}
```

Sample Input:

```
6
S1 60
S2 40
S1 80
S3 70
S2 90
S3 30
```

Sample Output:

S2 90.0

S1 70.0

S3 70.0

EXPERIMENT 3: Design and Implementation of a Maximum Profit Analyzer Using Kadane's Algorithm

Java Program:

```
package com.example;
import java.util.Scanner;
public class MaximumProfitAnalyzer {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 int[] arr = new int[n];
 for (int i = 0; i < n; i++) {
 arr[i] = sc.nextInt();
 }
 int currentSum = arr[0];
 int maxSum = arr[0];
 for (int i = 1; i < n; i++) {
 currentSum = Math.max(arr[i], currentSum + arr[i]);
 maxSum = Math.max(maxSum, currentSum);
}
 System.out.println(maxSum);
 sc.close();
 }
}
```

Sample Input:

8

-2 -3 4 -1 -2 1 5 -3

Sample Output:

7

EXPERIMENT 4: Design and Implementation of an Intelligent DNA Pattern Search Using KMP Algorithm

Java Program:

```
package com.example;
import java.util.*;
public class IntelligentDNAPatternSearch {
 // Function to build LPS array
 static int[] computeLPS(String pattern) {
 int m = pattern.length();
 int[] lps = new int[m];
 int len = 0;
 int i = 1;
 while (i < m) {
 if (pattern.charAt(i) == pattern.charAt(len)) {
 len++;
 lps[i] = len;
 i++;
 } else {
 if (len != 0) {
len = lps[len
- 1];
 } else {
 lps[i] = 0;
 i++;
```

```
}

}

}
 return lps;

}
 // KMP Search
 static void KMPSearch(String text, String pattern) {
 int[] lps = computeLPS(pattern);
 int i = 0;
 int j = 0;
 while (i < text.length()) {
 if (text.charAt(i) == pattern.charAt(j)) {
 i++;
 j++;

}
 if (j == pattern.length()) {
 System.out.print((i
- j) + " ");
 j = lps[j
- 1];

}
 else if (i < text.length() &&
 text.charAt(i) != pattern.charAt(j)) {
```

```
 if (j != 0)
 j = lps[j
- 1];
 else
 i++;

}

}

}
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 String text = sc.nextLine();
String pattern = sc.nextLine();
 KMPSearch(text, pattern);
 sc.close();
 }
}
```

Sample Input:

AABAACAADAABAABAAABA

Sample Output:

0 9 12

EXPERIMENT 5: Design and Implementation of a Banking Transaction System Using ObjectOriented Programming

Java Program:

```
package com.example;
import java.util.Scanner;
```

```
class BankAccount {
 int balance = 0;
 void deposit(int amount) {
 balance = balance + amount;
 }
 void withdraw(int amount) {
 balance = balance - amount;
 }
 void displayBalance() {
 System.out.println(balance);
 }
}
public class BankingTransactionSystem {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 BankAccount account = new BankAccount();
 int n = sc.nextInt();
 for (int i = 0; i < n; i++) {
 String operation = sc.next();
 int amount = sc.nextInt();
 if (operation.equalsIgnoreCase("Deposit")) {
 account.deposit(amount);
 }
 else if (operation.equalsIgnoreCase("Withdraw")) {
 account.withdraw(amount);
 }
 }
 account.displayBalance();
```

```
 sc.close();
 }
}
```

Sample Input:

3

Deposit 5000

Withdraw 2000

Deposit 1000

Sample Output:

4000

EXPERIMENT 6: Design and Implementation of a Ride Sharing Platform Simulator Using Object-Oriented Programming

Java Program:

```
package com.example;
import java.util.Scanner;
// Base class
class Vehicle {
 public int calculateFare(int distance) {
 return 0;
 }
}
// Bike class
class Bike extends Vehicle {
 public int calculateFare(int distance) {
 return distance * 5;
 }
}
// Auto class
```

```
class Auto extends Vehicle {
 public int calculateFare(int distance) {
 return distance * 12;
 }
}
// Cab class
class Cab extends Vehicle {
 public int calculateFare(int distance) {
 return distance * 12;
 }
}
public class RideSharingPlatformSimulator {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 for (int i = 0; i < n; i++) {
 String type = sc.next();
 int distance = sc.nextInt();
 Vehicle vehicle = null;
 try {
 if (type.equalsIgnoreCase("Bike")) {
 vehicle = new Bike();
 }
 else if (type.equalsIgnoreCase("Auto")) {
 vehicle = new Auto();
 }
 else if (type.equalsIgnoreCase("Cab")) {
 vehicle = new Cab();
```

```
}
else {
throw new IllegalArgumentException("Invalid Ride");
}
System.out.println(vehicle.calculateFare(distance));
} catch (IllegalArgumentException e) {
System.out.println(e.getMessage());
}
}
sc.close();
}
}
```

Sample Input:

3

Bike 10

Cab 15

Auto 8

Sample Output:

50

180

96

EXPERIMENT 7: Design and Implementation of a Hashtag Frequency Counter Using Maps

Java Program:

```
package com.example;
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;
```

```
public class HashtagFrequencyCounter {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 HashMap<String, Integer> hashtagCount = new HashMap<>();
 for (int i = 0; i < n; i++) {
 String hashtag = sc.next();
 if (hashtagCount.containsKey(hashtag)) {
 hashtagCount.put(hashtag, hashtagCount.get(hashtag) + 1);
 } else {
 hashtagCount.put(hashtag, 1);
 }
 }
for (Map.Entry<String, Integer> entry : hashtagCount.entrySet()) {
 System.out.println(entry.getKey() + " " + entry.getValue());
 }
 sc.close();
 }
}
```

Sample Input:

```
5
java
python
java
ai
python
```

Sample Output:

```
java 2
```

python 2

ai 1

EXPERIMENT 8: Design and Implementation of a Smart Traffic Navigation System Using Graph Representation

Java Program:

```
package com.example;
import java.util.*;
public class SmartTrafficNavigationSystem {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int N = sc.nextInt();
 int M = sc.nextInt();
 // Graph using adjacency list
 HashMap<Integer, ArrayList<Integer>> graph = new HashMap<>();
 for (int i = 1; i <= N; i++) {
 graph.put(i, new ArrayList<>());
 }
 // Read roads
 for (int i = 0; i < M; i++) {
 int u = sc.nextInt();
 int v = sc.nextInt();
graph.get(u).add(v);
 graph.get(v).add(u); // Undirected graph

}
 int source = sc.nextInt();
 int destination = sc.nextInt();
 Queue<Integer> queue = new LinkedList<>();
```

```
boolean[] visited = new boolean[N + 1];
queue.add(source);
visited[source] = true;
boolean found = false;
while (!queue.isEmpty()) {
int current = queue.poll();
if (current == destination) {
found = true;
break;

}
for (int neighbor : graph.get(current)) {
if (!visited[neighbor]) {
visited[neighbor] = true;
queue.add(neighbor);

}

}

}
if (found)
System.out.println("YES");
else
System.out.println("NO");
sc.close();

}
```

}

Sample Input

:

5 4

1 2

2 3

3 4

4 5

1 5

Sample Output:

YES

EXPERIMENT 9: Design and Implementation of Calculator Unit Testing Using Junit

JAVA PROGRAM:

Calculator.java:

```
package com.example;
public class Calculator {
 public int add(int a, int b) {
 return a + b;
 }
 public int divide(int a, int b) {
 return a / b;
 }
}
```

CalculatorTest.java (JUnit 5):

```
package com.example;
import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Test;
```

```
public class CalculatorTest {
 Calculator cal = new Calculator();
 @Test
void testAddition() {
 assertEquals(30, cal.add(10, 20));
 }
 @Test
 void testDivision() {
 assertEquals(5, cal.divide(10, 2));
 }
}
```

Sample Input:

10 + 20

Expected Test Result:

Test Passed

EXPERIMENT 10: Design and Implementation of Secure Authentication Stress Testing Using Automated Test Cases

Java Program:

```
package com.example;
import java.util.Scanner;
public class SecureAuthenticationStressTesting {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();
 for (int i = 0; i < n; i++) {
 String username = sc.next();
 String password = sc.next();
 if (username.length() >= 3 && username.length() <= 20 &&
```

```
password.length() >= 6 && password.length() <= 20) {
System.out.println("SUCCESS");
} else {
System.out.println("FAILURE");
}
}
sc.close();
}}
```

Sample Input:

```
3
admin admin123
ab 123
user pass
```

Sample Output:

```
SUCCESS
FAILURE
FAILURE
```

1.This Java 8 challenge tests your knowledge of Lambda expressions!

Write the following methods that *return a lambda expression* performing a specified action:

1. **PerformOperation isOdd(): The lambda expression must return if a number is odd or if it is even.**
2. **PerformOperation isPrime(): The lambda expression must return if a number is prime or if it is composite.**
3. **PerformOperation isPalindrome(): The lambda expression must return if a number is a palindrome or if it is not.**

Input Format

Input is handled for you by the locked stub code in your editor.

Output Format

The locked stub code in your editor will print lines of output.

Sample Input

The first line contains an integer, (the number of test cases).

The subsequent lines each describe a test case in the form of space-separated integers:
The first integer specifies the condition to check for (for Odd/Even, for Prime, or for Palindrome).
The second integer denotes the number to be checked.

5

1 4

2 5

3 898

1 3

2 12

Sample Output

EVEN

PRIME

PALINDROME

ODD

COMPOSITE

Program:

```
import java.io.*;

import java.util.*;
```

```
interface PerformOperation {
    boolean check(int a);
}

class MyMath {
    public static boolean checker(PerformOperation p, int num) {
        return p.check(num);
    }

    // 1. Checks if a number is odd
    public PerformOperation isOdd() {
        return n -> n % 2 != 0;
    }

    public PerformOperation isPrime() {
        return n -> {
            if (n < 2) return false;
            for (int i = 2; i <= Math.sqrt(n); i++) {
                if (n % i == 0) return false;
            }
            return true;
        };
    }


    public PerformOperation isPalindrome() {
        return n -> {
            String str = String.valueOf(n);
            String reversed = new StringBuilder(str).reverse().toString();
            return str.equals(reversed);
        };
```

```
    }
}

public class Solution {
    public static void main(String[] args) throws IOException {
        MyMath ob = new MyMath();
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        int T = Integer.parseInt(br.readLine());
        PerformOperation op;
        boolean ret = false;
        String ans = null;
        while (T-- > 0) {
            String s = br.readLine().trim();
            StringTokenizer st = new StringTokenizer(s);
            int ch = Integer.parseInt(st.nextToken());
            int num = Integer.parseInt(st.nextToken());
            if (ch == 1) {
                op = ob.isOdd();
                ret = ob.checker(op, num);
                ans = (ret) ? "ODD" : "EVEN";
            } else if (ch == 2) {
                op = ob.isPrime();
                ret = ob.checker(op, num);
                ans = (ret) ? "PRIME" : "COMPOSITE";
            } else if (ch == 3) {
                op = ob.isPalindrome();
                ret = ob.checker(op, num);
                ans = (ret) ? "PALINDROME" : "NOT PALINDROME";
            }
            System.out.println(ans);
        }
```

```
    }
}
```

Your Output (stdout)

```
EVEN
PRIME
PALINDROME
ODD
COMPOSITE
```

Expected Output

```
EVEN
PRIME
PALINDROME
ODD
```

2.You are given a list of student information: ID, FirstName, and CGPA. Your task is to rearrange them according to their CGPA in decreasing order. If two student have the same CGPA, then arrange them according to their first name in alphabetical order. If those two students also have the same first name, then order them according to their ID. No two students have the same ID.

- **Hint: You can use comparators to sort a list of objects. See the oracle docs to learn about comparators.**
- **Input Format**
- **The first line of input contains an integer , representing the total number of students. The next lines contains a list of student information in the following structure:**
- `ID Name CGPA`
- **The name contains only lowercase English letters. The contains only integer numbers without leading zeros. The *CGPA* will contain, at most, 2 digits after the decimal point.**
- **Output Format**

- **After rearranging the students according to the above rules, print the first name of each student on a separate line.**
- **Sample Input**

```
5
33 Rumpa 3.68
85 Ashis 3.85
56 Samiha 3.75
19 Samara 3.75
22 Fahim 3.76
```

- **Sample Output**

```
Ashis
Fahim
Samara
Samiha
Rumpa
```

Program

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;
import java.util.Scanner;

class Student {
    private int id;
    private String firstName;
    private double cgpa;

    public Student(int id, String firstName, double cgpa) {
        this.id = id;
        this.firstName = firstName;
        this.cgpa = cgpa;
    }
```

```
    public int getId() {
        return id;
    }

    public String getFirstName() {
        return firstName;
    }

    public double getCgpa() {
        return cgpa;
    }
}

public class Solution {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int n = scanner.nextInt();

        List<Student> studentList = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            int id = scanner.nextInt();
            String firstName = scanner.next();
            double cgpa = scanner.nextDouble();

            studentList.add(new Student(id, firstName, cgpa));
        }

        Collections.sort(studentList, new Comparator<Student>() {
            @Override
            public int compare(Student s1, Student s2) {
```

```
            // 1. CGPA in descending order
            if (Double.compare(s2.getCgpa(), s1.getCgpa()) != 0) {
                return Double.compare(s2.getCgpa(), s1.getCgpa());
            }



            if (!s1.getFirstName().equals(s2.getFirstName())) {
                return s1.getFirstName().compareTo(s2.getFirstName());
            }

            // 3. ID in ascending order
            return Integer.compare(s1.getId(), s2.getId());
        }
    });

    // Print the first name of each sorted student
    for (Student st : studentList) {
        System.out.println(st.getFirstName());
    }

    scanner.close();
  }
}
```

```
Input (stdin)
5
33 Rumpa 3.68
85 Ashis 3.85
56 Samiha 3.75
19 Samara 3.75
22 Fahim 3.76

Your Output (stdout)
Ashis
Fahim
Samara
Samiha
```

3. Given an integer array nums, move all the even integers at the beginning of the array followed by all the odd integers.

Return *any array that satisfies this condition*.

Example 1:

Input: nums = [3,1,2,4]

Output: [2,4,3,1]

Explanation: The outputs [4,2,3,1], [2,4,1,3], and [4,2,1,3] would also be accepted.

Example 2:

Input: nums = [0]

Output: [0]

Constraints:

- **1 <= nums.length <= 5000**
- **0 <= nums[i] <= 5000**

Program

```
class Solution {
    public int[] sortArrayByParity(int[] nums) {
        int left = 0;
        int right = nums.length - 1;

        while (left < right) {
            if (nums[left] % 2 > nums[right] % 2) {
                int temp = nums[left];
                nums[left] = nums[right];
                nums[right] = temp;
            }
            if (nums[left] % 2 == 0) left++;
            if (nums[right] % 2 == 1) right--;
        }
```

```
        return nums;
    }
}
```

Input

```
nums =
[3,1,2,4]
```

Output

```
[4,2,1,3]
```

Expected

```
[2,4,3,1]
```

4. Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array. The *Player* class is provided in the editor below. It has two fields:

1. **: a string.**
2. **: an integer.**

Given an array of *Player* objects, write a comparator that sorts them in order of decreasing score. If or more players have the same score, sort those players alphabetically ascending by name. To do this, you must create a *Checker* class that implements the *Comparator* interface, then write an *int compare(Player a, Player b)* method implementing the Comparator.compare(T o1, T o2) method. In short, when sorting in ascending order, a comparator function returns if , if , and if .

Declare a *Checker* class that implements the *comparator* method as described. It should sort first descending by score, then ascending by name. The code stub reads the input, creates a list of Player objects, uses your method to sort the data, and prints it out properly.

Example

Sort the list as . Sort first descending by score, then ascending by name.

Input Format

The first line contains an integer, , the number of players.
Each of the next lines contains a player's and , a string and an integer.

Constraints

-
- **Two or more players can have the same name.**
- **Player names consist of lowercase English alphabetic letters.**

Output Format

You are not responsible for printing any output to stdout. Locked stub code in *Solution* will instantiate a *Checker* object, use it to sort the *Player* array, and print each sorted element.

Sample Input

5

amy 100

david 100

heraldo 50

aakansha 75

aleksa 150

Sample Output

aleksa 150

amy 100

david 100

aakansha 75

heraldo 50

program:

```
import java.util.Arrays;
import java.util.Comparator;
import java.util.Scanner;


class Player {
   String name;
   int score;


   Player(String name, int score) {
      this.name = name;
      this.score = score;
   }
}
class Checker implements Comparator<Player> {
   @Override
```

```
    public int compare(Player a, Player b) {
        // 1. Sort by score in descending order
        if (a.score != b.score) {
            return Integer.compare(b.score, a.score);
        }



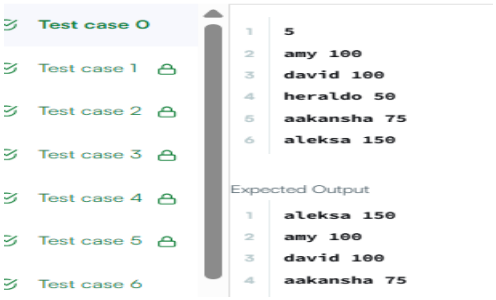
        return a.name.compareTo(b.name);
    }
}

public class Solution {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int n = scan.nextInt();

        Player[] player = new Player[n];
        Checker checker = new Checker();

        for (int i = 0; i < n; i++) {
            player[i] = new Player(scan.next(), scan.nextInt());
        }
        scan.close();

        Arrays.sort(player, checker);
        for (int i = 0; i < player.length; i++) {
            System.out.printf("%s %s\n", player[i].name, player[i].score);
        }
    }
}
```



5.Given an array nums. We define a running sum of an array as runningSum[i] = sum(nums[0]…nums[i]).

Return the running sum of nums.

Example 1:

Input: nums = [1,2,3,4]

Output: [1,3,6,10]

Explanation: Running sum is obtained as follows: [1, 1+2, 1+2+3, 1+2+3+4].

Example 2:

Input: nums = [1,1,1,1,1]

Output: [1,2,3,4,5]

Explanation: Running sum is obtained as follows: [1, 1+1, 1+1+1, 1+1+1+1, 1+1+1+1+1].

Example 3:

Input: nums = [3,1,2,10,1]

Output: [3,4,6,16,17]

Constraints:

- **1 <= nums.length <= 1000**
- **-10^6 <= nums[i] <= 10^6**

Program:

```
class Solution {
    public int[] runningSum(int[] nums) {
        for (int i = 1; i < nums.length; i++) {
            nums[i] += nums[i - 1];
        }
```

```
        return nums;
    }
```




```
}
```

6. You are given an m x n integer grid accounts where accounts[i][j] is the amount of money the ith customer has in the jth bank. Return *the wealth that the richest customer has.*

A customer's wealth is the amount of money they have in all their bank accounts. The richest customer is the customer that has the maximum wealth.

Example 1:

Input: accounts = [[1,2,3],[3,2,1]]

Output: 6

Explanation:

1st customer has wealth = 1 + 2 + 3 = 6

2nd customer has wealth = 3 + 2 + 1 = 6

Both customers are considered the richest with a wealth of 6 each, so return 6.

Example 2:

Input: accounts = [[1,5],[7,3],[3,5]]

Output: 10

Explanation:

1st customer has wealth = 6

2nd customer has wealth = 10

3rd customer has wealth = 8

The 2nd customer is the richest with a wealth of 10.

Example 3:

Input: accounts = [[2,8,7],[7,1,3],[1,9,5]]

Output: 17

Constraints:

- **m == accounts.length**
- **n == accounts[i].length**
- **1 <= m, n <= 50**
- **1 <= accounts[i][j] <= 100**

Program:

```
class Solution {
    public int maximumWealth(int[][] accounts) {
        int maxWealth = 0;

        for (int[] customer : accounts) {
            int currentWealth = 0;
            for (int bank : customer) {
                currentWealth += bank;
            }
            maxWealth = Math.max(maxWealth, currentWealth);
        }

        return maxWealth;
    }
}
```

7.Given an integer array nums sorted in non-decreasing order, return *an array of the squares of each number sorted in non-decreasing order*.

Example 1:

Input: nums = [-4,-1,0,3,10]

Output: [0,1,9,16,100]

Explanation: After squaring, the array becomes [16,1,0,9,100].

After sorting, it becomes [0,1,9,16,100].

Example 2:

Input: nums = [-7,-3,2,3,11]

Output: [4,9,9,49,121]

Constraints:

- **1 <= nums.length <= 10^4**
- **-10^4 <= nums[i] <= 10^4**
- **nums is sorted in non-decreasing order**.

```
class Solution {
    public int[] sortedSquares(int[] nums) {
        int n = nums.length;
        int[] result = new int[n];

        int left = 0;
        int right = n - 1;
        int pos = n - 1; // Fill the result array from back to front

        while (left <= right) {
            int leftSquare = nums[left] * nums[left];
            int rightSquare = nums[right] * nums[right];

            if (leftSquare > rightSquare) {
                result[pos] = leftSquare;
                left++;
            } else {
                result[pos] = rightSquare;
                right--;
            }
```

```
            pos--;
        }

        return result;
    }
}
```

Input

```
nums =
[1,2,3,4]
```

Output

```
[1,3,6,10]
```

Expected

```
[1,3,6,10]
```

8.Given an array of integers nums, calculate the pivot index of this array.

The pivot index is the index where the sum of all the numbers strictly to the left of the index is equal to the sum of all the numbers strictly to the index's right.

If the index is on the left edge of the array, then the left sum is 0 because there are no elements to the left. This also applies to the right edge of the array.

Return *the leftmost pivot index*. If no such index exists, return -1.

Example 1:

Input: nums = [1,7,3,6,5,6]

Output: 3

Explanation:

The pivot index is 3.

Left sum = nums[0] + nums[1] + nums[2] = 1 + 7 + 3 = 11

Right sum = nums[4] + nums[5] = 5 + 6 = 11

Example 2:

Input: nums = [1,2,3]

Output: -1

Explanation:

There is no index that satisfies the conditions in the problem statement.

Example 3:

Input: nums = [2,1,-1]

Output: 0

Explanation:

The pivot index is 0.

Left sum = 0 (no elements to the left of index 0)

Right sum = nums[1] + nums[2] = 1 + -1 = 0

Constraints:

- **1 <= nums.length <= 10^4**
- **-1000 <= nums[i] <= 1000**

- **program**

```
class Solution {
    public int pivotIndex(int[] nums) {
        int totalSum = 0;
        for (int num : nums) {
            totalSum += num;
        }

        int leftSum = 0;
        for (int i = 0; i < nums.length; i++) {
            // Right sum is totalSum - leftSum - nums[i]
            if (leftSum == totalSum - leftSum - nums[i]) {
                return i; // Found the leftmost pivot index
            }
            leftSum += nums[i];
        }

        return -1;
    }
}
```

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums =

```
[1,7,3,6,5,6]
```

Output

```
3
```

Expected

```
3
```

1.Given a zero-based permutation nums (0-indexed), build an array ans of the same length where ans[i] = nums[nums[i]] for each 0 <= i < nums.length and return it.

A zero-based permutation nums is an array of distinct integers from 0 to nums.length - 1 (inclusive).

Example 1:

Input: nums = [0,2,1,5,3,4]

Output: [0,1,2,4,5,3]

Explanation: The array ans is built as follows:

ans = [nums[nums[0]], nums[nums[1]], nums[nums[2]], nums[nums[3]], nums[nums[4]], nums[nums[5]]]

= [nums[0], nums[2], nums[1], nums[5], nums[3], nums[4]]

= [0,1,2,4,5,3]

Example 2:

Input: nums = [5,0,1,2,3,4]

Output: [4,5,0,1,2,3]

Explanation: The array ans is built as follows:

ans = [nums[nums[0]], nums[nums[1]], nums[nums[2]], nums[nums[3]], nums[nums[4]], nums[nums[5]]]

= [nums[5], nums[0], nums[1], nums[2], nums[3], nums[4]]

= [4,5,0,1,2,3]

Constraints:

- **1 <= nums.length <= 1000**
- **0 <= nums[i] < nums.length**
- **The elements in nums are distinct.**

Program:

```
public class Solution {
    public int[] buildArray(int[] nums) {
```

```
        int n = nums.length;

        for (int i = 0; i < n; i++) {
            nums[i] = nums[i] + n * (nums[nums[i]] % n);
        }

        for (int i = 0; i < n; i++) {
            nums[i] /= n;
        }

        return nums;
    }
}
```

Input

```
nums =
[0,2,1,5,3,4]
```

Output

```
[0,1,2,4,5,3]
```

Expected

```
[0,1,2,4,5,3]
```

2. Given the array nums consisting of 2n elements in the form $[x_1,x_2,...,x_n,y_1,y_2,...,y_n]$.

Return the array in the form **$[x_1,y_1,x_2,y_2,...,x_n,y_n]$.**

Example 1:

Input: nums = [2,5,1,3,4,7], n = 3

Output: [2,3,5,4,1,7]

Explanation: Since x_1=2, x_2=5, x_3=1, y_1=3, y_2=4, y_3=7 then the answer is [2,3,5,4,1,7].

Example 2:

Input: nums = [1,2,3,4,4,3,2,1], n = 4

Output: [1,4,2,3,3,2,4,1]

Example 3:

Input: nums = [1,1,2,2], n = 2

Output: [1,2,1,2]

Constraints:

- **1 <= n <= 500**
- **nums.length == 2n**
- **1 <= nums[i] <= 10^3**

Program:

```
public class Solution {
    public int[] shuffle(int[] nums, int n) {
        int[] ans = new int[2 * n];
        for (int i = 0; i < n; i++) {
            ans[2 * i] = nums[i];       // Place x_i
            ans[2 * i + 1] = nums[i + n]; // Place y_i
        }
        return ans;
    }
}
```

Input

nums =
[2,5,1,3,4,7]

n =
3

Output

[2,3,5,4,1,7]

Expected

[2,3,5,4,1,7]

3.Given an integer array nums and an integer val, remove all occurrences of val in nums in-place. The order of the elements may be changed. Then return *the number of elements in* nums *which are not equal to* val.

Consider the number of elements in nums which are not equal to val be k, to get accepted, you need to do the following things:

- **Change the array nums such that the first k elements of nums contain the elements which are not equal to val. The remaining elements of nums are not important as well as the size of nums.**
- **Return k.**

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int val = ...; // Value to remove
int[] expectedNums = [...]; // The expected answer with correct length.
                            // It is sorted with no values equaling val.

int k = removeElement(nums, val); // Calls your implementation

assert k == expectedNums.length;
sort(nums, 0, k); // Sort the first k elements of nums
for (int i = 0; i < actualLength; i++) {
    assert nums[i] == expectedNums[i];
```

}

If all assertions pass, then your solution will be accepted.

Example 1:

Input: nums = [3,2,2,3], val = 3

Output: 2, nums = [2,2,_,_]

Explanation: Your function should return k = 2, with the first two elements of nums being 2.

It does not matter what you leave beyond the returned k (hence they are underscores).

Example 2:

Input: nums = [0,1,2,2,3,0,4,2], val = 2

Output: 5, nums = [0,1,4,0,3,_,_,_]

Explanation: Your function should return k = 5, with the first five elements of nums containing 0, 0, 1, 3, and 4.

Note that the five elements can be returned in any order.

It does not matter what you leave beyond the returned k (hence they are underscores).

Constraints:

- **0 <= nums.length <= 100**
- **0 <= nums[i] <= 50**
- **0 <= val <= 100**

Program:

```
public class Solution {
    public int removeElement(int[] nums, int val) {
        int k = 0;
```

```
        for (int i = 0; i < nums.length; i++) {
            if (nums[i] != val) {
                nums[k] = nums[i];
                k++;
            }
        }
        return k;
    }


}
```

Input

nums =
[3,2,2,3]

val =
3

Output

[2,2]

Expected

[2,2]

4. Given an integer array nums sorted in non-decreasing order, remove the duplicates in-place such that each unique element appears only once. The relative order of the elements should be kept the same.

Consider the number of *unique elements* in nums to be k. After removing duplicates, return the number of unique elements k.

The first k elements of nums should contain the unique numbers in sorted order. The remaining elements beyond index k - 1 can be ignored.

Custom Judge:

The judge will test your solution with the following code:

int[] nums = [...]; // Input array

int[] expectedNums = [...]; // The expected answer with correct length

int k = removeDuplicates(nums); // Calls your implementation

assert k == expectedNums.length;

for (int i = 0; i < k; i++) {

assert nums[i] == expectedNums[i];

}

If all assertions pass, then your solution will be accepted.

Example 1:

Input: nums = [1,1,2]

Output: 2, nums = [1,2,_]

Explanation: Your function should return k = 2, with the first two elements of nums being 1 and 2 respectively.

It does not matter what you leave beyond the returned k (hence they are underscores).

Example 2:

Input: nums = [0,0,1,1,1,2,2,3,3,4]

Output: 5, nums = [0,1,2,3,4,_,_,_,_,_]

Explanation: Your function should return k = 5, with the first five elements of nums being 0, 1, 2, 3, and 4 respectively.

It does not matter what you leave beyond the returned k (hence they are underscores).

Constraints:

- **$1 <= nums.length <= 3 * 10^4$**
- **-100 <= nums[i] <= 100**

- **nums is sorted in non-decreasing order.**

Program:

```
public class Solution {
    public int removeDuplicates(int[] nums) {
        if (nums.length == 0) return 0;

        int k = 1; // Pointer for placing the next unique element
        for (int i = 1; i < nums.length; i++) {
            if (nums[i] != nums[i - 1]) {
                nums[k] = nums[i];
                k++;
            }
        }
        return k;
    }
}
```



5.Given an integer array nums, find the subarray with the largest sum, and return *its sum*.

Example 1:

Input: nums = [-2,1,-3,4,-1,2,1,-5,4]

Output: 6

Explanation: The subarray [4,-1,2,1] has the largest sum 6.

Example 2:

Input: nums = [1]

Output: 1

Explanation: The subarray [1] has the largest sum 1.

Example 3:

Input: nums = [5,4,-1,7,8]

Output: 23

Explanation: The subarray [5,4,-1,7,8] has the largest sum 23.

Constraints:

- **$1 <= nums.length <= 10^5$**
- **$-10^4 <= nums[i] <= 10^4$**

Program:

```
public class Solution {
    public int maxSubArray(int[] nums) {
        int maxSum = nums[0];
        int currentSum = nums[0];

        for (int i = 1; i < nums.length; i++) {
            currentSum = Math.max(nums[i], currentSum + nums[i]);
            maxSum = Math.max(maxSum, currentSum);
        }
```

```
        return maxSum;
    }
}
```

Input

```
nums =
[-2,1,-3,4,-1,2,1,-5,4]
```

Output

```
6
```

Expected

```
6
```

6. There is a biker going on a road trip. The road trip consists of n + 1 points at various altitudes. The biker starts his trip on point 0 with altitude equal 0.

You are given an integer array gain of length n where gain[i] is the net gain in altitude between points i and i + 1 for all (0 <= i < n). Return *the highest altitude of a point.*

Example 1:

Input: gain = [-5,1,5,0,-7]

Output: 1

Explanation: The altitudes are [0,-5,-4,1,1,-6]. The highest is 1.

Example 2:

Input: gain = [-4,-3,-2,-1,4,3,2]

Output: 0

Explanation: The altitudes are [0,-4,-7,-9,-10,-6,-3,-1]. The highest is 0.

Constraints:

- **n == gain.length**
- **1 <= n <= 100**

- **-100 <= gain[i] <= 100**

Program:

```
Solution public class {
    public int largestAltitude(int[] gain) {
        int maxAltitude = 0;
        int currentAltitude = 0;

        for (int g : gain) {
            currentAltitude += g;
            maxAltitude = Math.max(maxAltitude, currentAltitude);
        }

        return maxAltitude;
    }
}
```

Input

gain =
[-5,1,5,0,-7]

Output

1

Expected

1

7. Given an array of strings strs, group the anagrams together. You can return the answer in any order.

Example 1:

Input: strs = ["eat","tea","tan","ate","nat","bat"]

Output: [["bat"],["nat","tan"],["ate","eat","tea"]]

Explanation:

- **There is no string in strs that can be rearranged to form "bat".**
- **The strings "nat" and "tan" are anagrams as they can be rearranged to form each other.**
- **The strings "ate", "eat", and "tea" are anagrams as they can be rearranged to form each other.**

Example 2:

Input: strs = [""]

Output: [[""]]

Example 3:

Input: strs = ["a"]

Output: [["a"]]

Constraints:

- **1 <= strs.length <= 10^4**
- **0 <= strs[i].length <= 100**
- **strs[i] consists of lowercase English letters.**

Program:

```
import java.util.*;


public class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {
        Map<String, List<String>> map = new HashMap<>();


        for (String s : strs) {
            char[] chars = s.toCharArray();
            Arrays.sort(chars);
```

```
            String key = new String(chars);

            map.putIfAbsent(key, new ArrayList<>());
            map.get(key).add(s);
        }

        return new ArrayList<>(map.values());
    }
}
```

Input

```
strs =
["eat","tea","tan","ate","nat","bat"]
```

Output

```
[["eat","tea","ate"],["bat"],["tan","nat"]]
```

Expected

```
[["bat"],["nat","tan"],["ate","eat","tea"]]
```

8. Given an integer array nums and an integer k, return *the* k *most frequent elements*. You may return the answer in any order.

Example 1:

Input: nums = [1,1,1,2,2,3], k = 2

Output: [1,2]

Example 2:

Input: nums = [1], k = 1

Output: [1]

Example 3:

Input: nums = [1,2,1,2,1,2,3,1,3,2], k = 2

Output: [1,2]

Constraints:

- **1 <= nums.length <= 10^5**
- **-10^4 <= nums[i] <= 10^4**
- **k is in the range [1, the number of unique elements in the array].**
- **It is guaranteed that the answer is unique.**

Program:

```
import java.util.*;


public class Solution {
    public int[] topKFrequent(int[] nums, int k) {
        Map<Integer, Integer> count = new HashMap<>();
        for (int num : nums) {
            count.put(num, count.getOrDefault(num, 0) + 1);
        }

        List<Integer>[] buckets = new List[nums.length + 1];
        for (int key : count.keySet()) {
            int freq = count.get(key);
            if (buckets[freq] == null) {
                buckets[freq] = new ArrayList<>();
            }
            buckets[freq].add(key);
        }
```

```
        int[] result = new int[k];
        int index = 0;
        for (int i = buckets.length - 1; i >= 0 && index < k; i--) {
            if (buckets[i] != null) {
                for (int num : buckets[i]) {
                    result[index++] = num;
                    if (index == k) break;
                }
            }
        }
        return result;
    }
}
```

Input

```
nums =
[1,1,1,2,2,3]
```

```
k =
2
```

Output

```
[1,2]
```

Expected

```
[1,2]
```

9. In computer science, a double-ended queue (dequeue, often abbreviated to deque, pronounced deck) is an abstract data type that generalizes a queue, for which elements can be added to or removed from either the front (head) or back (tail).

Deque interfaces can be implemented using various types of collections such as LinkedList or ArrayDeque classes. For example, deque can be declared as:

Deque deque = new LinkedList<>();

or

Deque deque = new ArrayDeque<>();

You can find more details about Deque here.

In this problem, you are given integers. You need to find the maximum number of unique integers among all the possible contiguous subarrays of size .

***Note*: Time limit is second for this problem.**

Input Format

The first line of input contains two integers and : representing the total number of integers and the size of the subarray, respectively. The next line contains space separated integers.

Constraints

The numbers in the array will range between .

Output Format

Print the *maximum* number of unique integers among all possible contiguous subarrays of size .

Sample Input

6 3

5 3 5 2 3 2

Sample Output

3

Program:

```
import java.util.*;


public class test {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
```

```
Deque<Integer> deque = new ArrayDeque<>();
Map<Integer, Integer> map = new HashMap<>();

int n = in.nextInt();
int m = in.nextInt();
int maxUnique = 0;

for (int i = 0; i < n; i++) {
   int num = in.nextInt();



   deque.add(num);
   map.put(num, map.getOrDefault(num, 0) + 1);



   if (deque.size() == m) {
      maxUnique = Math.max(maxUnique, map.size());



      if (maxUnique == m) {
         System.out.println(maxUnique);
         return;
      }



      int oldVal = deque.removeFirst();
```

```
            if (map.get(oldVal) == 1) {
                map.remove(oldVal);
            } else {
                map.put(oldVal, map.get(oldVal) - 1);
            }
        }
    }

    System.out.println(maxUnique);
  }
}
```

Input (stdin)

```
6 3
5 3 5 2 3 2
```

Your Output (stdout)

```
3
```

Expected Output

```
3
```

10.in computer science, a set is an abstract data type that can store certain values, without any particular order, and no repeated values(Wikipedia). is an example of a set, but is not a set. Today you will learn how to use sets in java by solving this problem.

You are given pairs of strings. Two pairs and are identical if and . That also implies is *not* same as . After taking each pair as input, you need to print number of unique pairs you currently have.

Complete the code in the editor to solve this problem.

Input Format

In the first line, there will be an integer denoting number of pairs. Each of the next lines will contain two strings seperated by a single space.

Constraints:

- Length of each string is atmost and will consist lower case letters only.

Output Format

Print lines. In the line, print number of unique pairs you have after taking pair as input.

Sample Input

```
5
```

```
john tom
john mary
john tom
mary anna
mary anna
```

Sample Output

```
1
2
2
3
3
```

Program:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

public class Solution {


 public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        int t = s.nextInt();
        String [] pair_left = new String[t];
        String [] pair_right = new String[t];

        for (int i = 0; i < t; i++) {
            pair_left[i] = s.next();
            pair_right[i] = s.next();
        }
 HashSet<String> set = new HashSet<>(t * 2);
        StringBuilder sb = new StringBuilder();

        for (int i = 0; i < t; i++) {
```

```
            set.add(pair_left[i] + " " + pair_right[i]);
            sb.append(set.size()).append("\n");
        }

        System.out.print(sb);
    }
}
```

Your Output (stdout)

```
1
2
2
3
3
```

Expected Output

```
1
2
2
3
```

WEEK 3

TASK 3.1 : JAVA DATE AND TIME

PROGRAM:

```
class Result {


    public static String findDay(int month, int day, int year) {
        java.util.Calendar cal = java.util.Calendar.getInstance
        cal.set(year, month - 1,day);
        return cal.getDisplayName(
            java.util.Calendar.DAY_OF_WEEK,
            java.util.Calendar.LONG,
            Locale.ENGLISH
        ).toUpperCase();


    }


}
```

OUTPUT:

Input (stdin)

```
08 05 2015
```

Expected Output

```
WEDNESDAY
```

TASK 3.2: NUMBER OF DAYS BETWEEN TWO DAYS

PROGRAM:

```
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;

class Solution {
    public int daysBetweenDates(String date1, String date2) {

        LocalDate d1 = LocalDate.parse(date1);
        LocalDate d2 = LocalDate.parse(date2);
        return (int) Math. Abs(ChronoUnit.DAYS.between(d1,d2));
    }
}
```

OUTPUT:



TASK 3.3:DAY OF THE YEAR

PROGRAM:

```
import java.time.LocalDate;

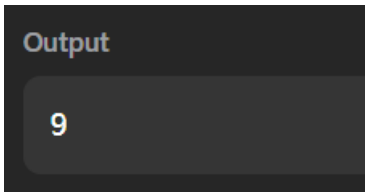
class Solution {
    public int dayOfYear(String date) {

        LocalDate d = LocalDate.parse(date);

        return d.getDayOfYear();

    }
}
```

OUTPUT:



TASK 3.4: DAY OF THE WEEK

PROGRAM:

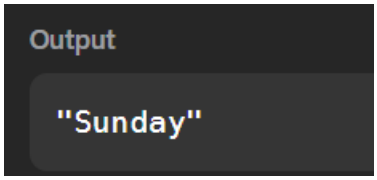
```
import java.time.LocalDate;
import java.time.format.TextStyle;
import java.util.Locale;


class Solution {
    public String dayOfTheWeek(int day, int month, int year) {


        LocalDate date = LocalDate.of(year,month, day);


        return date.getDayOfWeek()
                .getDisplayName(TextStyle.FULL, Locale.ENGLISH);
    }
}
```

OUTPUT:



TASK 3.5: JAVA PRIORITY QUEUE

PROGRAM:

```
import java.util.*;
class Student {
    private int id;
    private String name;
    private double cgpa;
    public Student(int id, String name, double cgpa) {
        this.id = id;
        this.name = name;
        this.cgpa = cgpa;
    }
    public int getID() {
        return id;
    }
    public String getName() {
        return name;
    }
    public double getCGPA() {
        return cgpa;
    }
}
class Priorities {
    public List<Student> getStudents(List<String> events) {
        PriorityQueue<Student> pq = new PriorityQueue<>(
            (s1, s2) ->
                if (s1.getCGPA() != s2.getCGPA()) {
                    return Double.compare(
                        s2.getCGPA(),
                        s1.getCGPA()
```

```
            );
        }
        int nameCompare =
            s1. getName().compareTo(s2.getName());
        if (nameCompare != 0) {
            return nameCompare;
        }
        return Integer.compare(
            s1.getID(),
            s2.getID()
        );
    }
);
for (String event : events) {
    if (event.startsWith("ENTER")) {
        String[] parts = event.split(" ");
        String name = parts[1];
        double cgpa = Double.parseDouble(parts[2]);
        int id = Integer.parseInt(parts[3]);
        pq.add(new Student(id, name, cgpa));
    } else if (event.equals("SERVED")) {
        if (!pq.isEmpty()) {
            pq.poll();
        }
    }
}
List<Student> result = new ArrayList<>();
while (!pq.isEmpty()) {
    result.add(pq.poll());
}
```

```
        return result;
    }
}
public class Solution {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        sc.nextLine();
        List<String> events = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            events.add(sc.nextLine());
        }
        Priorities priorities = new Priorities();
        List<Student> students =
            priorities.getStudents(events);
        if (students.isEmpty()) {
            System.out.println("EMPTY");
        } else {
            for (Student student : students) {
                System.out.println(student.getName());
            }
        }

        sc.close();
    }
}
```

OUTPUT:

Expected Output

```
Dan
Ashley
Shafaet
Maria
```

TASK 3.6:JAVA ARRAYLIST

Program:

```
import java.io.*;
import java.util.*;
public class Solution {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        ArrayList<ArrayList<Integer>> list = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            int d = sc.nextInt();
            ArrayList<Integer> numbers = new ArrayList<>();
            for (int j = 0; j < d; j++) {
                numbers.add(sc.nextInt());
            }
            list.add(numbers);
        }

        int q = sc.nextInt();
        for (int i = 0; i < q; i++) {
            int x = sc.nextInt();
            int y = sc.nextInt();
            try {
                System.out.println(list.get(x - 1).get(y - 1));
            }
            catch (IndexOutOfBoundsException e) {
                System.out.println("ERROR!");
            }
        }
```

```
        sc.close();
    }
}
```

OUTPUT:

Expected Output

```
74
52
37
ERROR!
ERROR!
```

TASK 3.7:LARGEST NUMBER

PROGRAM:

```
class Solution {
    public String largestNumber(int[] nums) {
        String[] arr= new String[nums.length];
        for (int i = 0;i < nums.length; i++) {
            arr[i] = nums[i] + "";
        }
        Arrays.sort(arr, (a,b) -> (b + a).compareTo(a+b));
        if (arr[0].equals("0")) {
            return "0";
        }
        String answer = "";
        for (String s: arr) {
            answer = answer + s;
        }
        return answer;

    }
}
```

OUTPUT:



TASK 3.8:JAVA COMPARATOR

PROGRAM:

```
import java.util.*;
class Checker implements Comparator<Player> {
    public int compare(Player a, Player b) {
        if (a.score != b.score) {
            return b.score - a.score;
        }
        return a.name.compareTo(b.name);
    }
}
class Player{
    String name;
    int score;
    Player(String name, int score){
        this.name = name;
        this.score = score;
    }
}
class Solution {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int n = scan.nextInt();
        Player[] player = new Player[n];
        Checker checker = new Checker();
        for(int i = 0; i < n; i++){
            player[i] = new Player(scan.next(), scan.nextInt());
        }
        scan.close();
```

```
        Arrays.sort(player, checker);
        for(int i = 0; i < player. Length; i++){
            System.out.printf("%s %s\n", player[i].name, player[i].score);
        }
    }
}
```

OUTPUT:

Expected Output

```
aleksa 150
amy 100
david 100
aakansha 75
heraldo 50
```

TASK 3.9:SORT THE PEOPLE

PROGRAM:

```
import java.util.*;
class Solution {
    public String[] sortPeople(String[] names, int[] heights) {
        Integer[] index = new Integer[names.length];
        for (int i = 0; i < names.length; i++) {
            index[i] = i;
        }
        Arrays.sort(index, (a, b) -> heights[b] - heights[a]);
        String[] result = new String[names.length];
        for (int i = 0; i < names.length; i++) {
            result[i] = names[index[i]];
        }
        return result;
    }
}
```

OUTPUT:



WEEK – 4

TASK 4.1: DETERMINE-IF-STRING-HALVES-ARE-LIKE

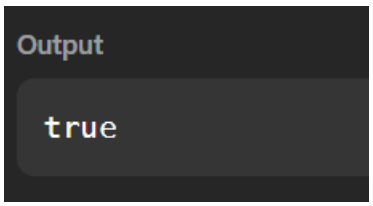
PROGRAM:

```
class Solution {
    public boolean halvesAreAlike(String s) {
        int mid = s.length() / 2;
        long first = s.substring(0, mid)
                .chars()
                .filter(c -> "aeiouAEIOU".indexOf(c) != -1)
                .count();

        long second = s.substring(mid)
                .chars()
                .filter(c -> "aeiouAEIOU".indexOf(c) != -1)
                .count();

        return first == second;
    }
}
```

OUTPUT:



TASK 4.2 : LAPIN

PROGRAM:

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int T = sc.nextInt();
        while (T-- > 0) {
            String s = sc.next();
            int n = s.length();
            int mid = n / 2;
            int[] first = new int[26];
            int[] second = new int[26];
            for (int i = 0; i < mid; i++) {
                first[s.charAt(i) - 'a']++;
            }
            for (int i = (n + 1) / 2; i < n; i++) {
                second[s.charAt(i) - 'a']++;
            }
            if (Arrays.equals(first, second)) {
                System.out.println("YES");
            } else {
                System.out.println("NO");
            }
        }
        sc.close();
    }
}
```

OUTPUT:

Your Output

```
YES
NO
YES
YES
NO
NO
```

TASK 4.3 : COMPARE THE TRIPLETS

PROGRAM:

```
import java.util.*;
class Result {
    public static List<Integer> compareTriplets(List<Integer> a, List<Integer> b) {
        int aliceScore = 0;
        int bobScore = 0;
        for (int i = 0; i < 3; i++) {
            if (a.get(i) > b.get(i)) {
                aliceScore++;
            }
            else if (a.get(i) < b.get(i)) {
                bobScore++;
            }
        }
        return Arrays.asList(aliceScore, bobScore);
    }
}
public class Solution {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        List<Integer> a = new ArrayList<>();
        List<Integer> b = new ArrayList<>();
        for (int i = 0; i < 3; i++) {
            a.add(sc.nextInt());
        }
        for (int i = 0; i < 3; i++) {
            b.add(sc.nextInt());
        }
```

```
        List<Integer> result = Result.compareTriplets(a, b);
        for (int score : result) {
            System.out.print(score + " ");
        }
        sc.close();
    }
}
```

OUTPUT:

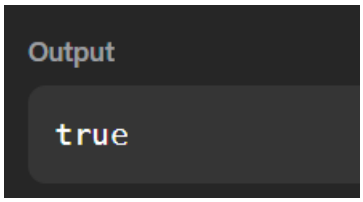


TASK 4.4 : CONTAINS DUPLICATES

PROGRAM:

```
class Solution {
    public boolean containsDuplicate(int[] nums) {
        HashSet<Integer> set = new HashSet<>();
        for (int n : nums) {
            if(set.contains(n)) {
                return true;
            }
            set.add(n);
        }
        return false;
    }
}
```

OUTPUT:



TASK 4.5 : TIME CONVERSION

PROGRAM:

```
import java.io.*;
import java.util.*;
class Result {
    public static String timeConversion(String s) {
        String period = s.substring(8, 10);
        int hour = Integer.parseInt(s.substring(0, 2));
        if (period.equals("AM")) {
            if (hour == 12) {
                hour = 0;
            }
        }
        else {
            if (hour != 12) {
                hour = hour + 12;
            }
        }
        return String.format("%02d", hour) + s.substring(2, 8);
    }
}
public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader =
            new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter =
            new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));
        String s = bufferedReader.readLine();
        String result = Result.timeConversion(s);
```

```
        bufferedWriter.write(result);
        bufferedWriter.newLine();
        bufferedReader.close();
        bufferedWriter.close();
    }
}
```

OUTPUT:

Expected Output

```
19:05:45
```

TASK 4.6 :MOVE ZEROES

PROGRAM:

```
class Solution {
    public void moveZeroes(int[] nums) {

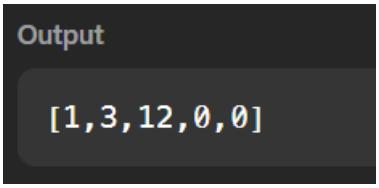
        int j = 0;

        for (int i = 0; i < nums.length; i++) {

            if (nums[i] != 0) {
                int temp = nums[i];
                nums[i] = nums[j];
                nums[j] = temp;

                j++;
            }
        }
    }
}
```

OUTPUT:



TASK 4.7:DIAGONAL DIFFERENCE

PROGRAM:

```
import java.util.*;

public class Solution {

    public static int diagonalDifference(List<List<Integer>> arr) {
        int n = arr.size();
        int leftDiagonal = 0;
        int rightDiagonal = 0;

        for (int i = 0; i < n; i++) {
            leftDiagonal += arr.get(i).get(i);
            rightDiagonal += arr.get(i).get(n - 1 - i);
        }

        return Math.abs(leftDiagonal - rightDiagonal);
    }
}
```

OUTPUT:



TASK 4.8: TRANSPOSE MATRIX

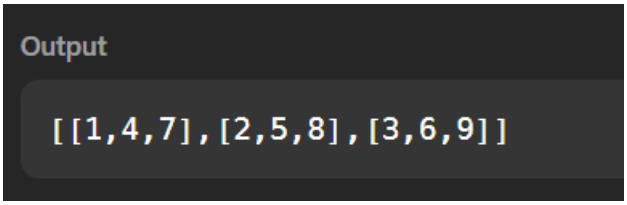
PROGRAM:

```
import java.util.stream.IntStream;

class Solution {
    public int[][] transpose(int[][] matrix) {
        int rows = matrix.length;
        int cols = matrix[0].length;

        return IntStream.range(0, cols)
                .mapToObj(j ->
                        IntStream.range(0,rows)
                                .map(i -> matrix[i][j])
                                .toArray()
                )
                .toArray(int[][]::new);
    }
}
```

OUTPUT:



TASK 4.9:MATRIX BLOCK SUM

PROGRAM:

```
class Solution {
    public int[][] matrixBlockSum(int[][] mat, int k) {

        int m = mat.length;
        int n = mat[0].length;

        int[][] answer = new int[m][n];

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {

                int sum = 0;

                for (int r = Math.max(0, i - k);
                    r <= Math.min(m - 1, i + k);
                    r++) {

                    for (int c = Math.max(0, j - k);
                        c <= Math.min(n - 1, j + k);
                        c++) {

                        sum += mat[r][c];
                    }
                }

                answer[i][j] = sum;
            }
        }

        return answer;
    }
}
```

OUTPUT:



TASK 4.10: MATRIX ROTATION

PROGRAM:

```
import java.io.*;
import java.util.*;

public class Solution {

    public static void matrixRotation(List<List<Integer>> matrix, int r) {

        int rows = matrix.size();
        int cols = matrix.get(0).size();

        int[][] result = new int[rows][cols];

        for (int layer = 0; layer < Math.min(rows, cols) / 2; layer++) {

            List<Integer> elements = new ArrayList<>();

            for (int j = layer; j < cols - layer; j++) {
                elements.add(matrix.get(layer).get(j));
            }

            for (int i = layer + 1; i < rows - layer; i++) {
                elements.add(matrix.get(i).get(cols - layer - 1));
            }
            for (int j = cols - layer - 2; j >= layer; j--) {
                elements.add(matrix.get(rows - layer - 1).get(j));
            }


            for (int i = rows - layer - 2; i > layer; i--) {
                elements.add(matrix.get(i).get(layer));
            }

            int size = elements.size();
            int rotation = r % size;
            int index = 0;
            for (int j = layer; j < cols - layer; j++) {
                result[layer][j] =
                    elements.get((index + rotation) % size);
                index++;
            }
            for (int i = layer + 1; i < rows - layer; i++) {
                result[i][cols - layer - 1] =
                    elements.get((index + rotation) % size);
                index++;
            }
            for (int j = cols - layer - 2; j >= layer; j--) {
                result[rows - layer - 1][j] =
                    elements.get((index + rotation) % size);
                index++;
            }
```

```
            for (int i = rows - layer - 2; i > layer; i--) {
                result[i][layer] =
                    elements.get((index + rotation) % size);
                index++;
            }
        }

        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                System.out.print(result[i][j]);

                if (j < cols - 1) {
                    System.out.print(" ");
                }
            }
            System.out.println();
        }
    }

    public static void main(String[] args) throws IOException {

        BufferedReader br =
            new BufferedReader(new InputStreamReader(System.in));

        String[] firstLine = br.readLine().trim().split(" ");

        int rows = Integer.parseInt(firstLine[0]);
        int cols = Integer.parseInt(firstLine[1]);
        int r = Integer.parseInt(firstLine[2]);

        List<List<Integer>> matrix = new ArrayList<>();

        for (int i = 0; i < rows; i++) {

            String[] values = br.readLine().trim().split(" ");

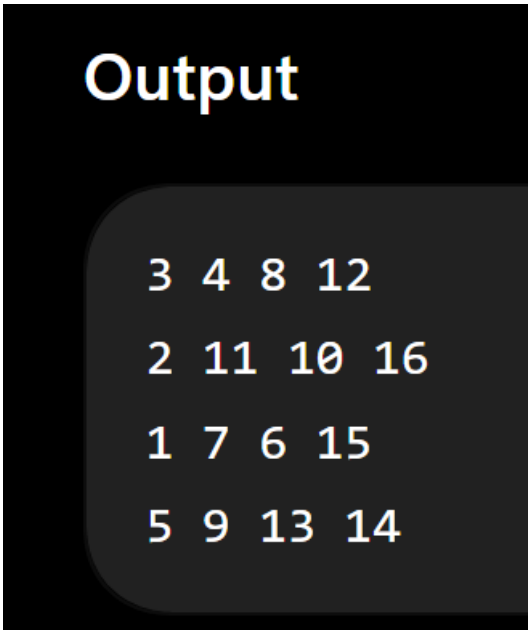
            List<Integer> row = new ArrayList<>();

            for (String value : values) {
                row.add(Integer.parseInt(value));
            }

            matrix.add(row);
        }

        matrixRotation(matrix, r);
    }
}
```

OUTPUT:



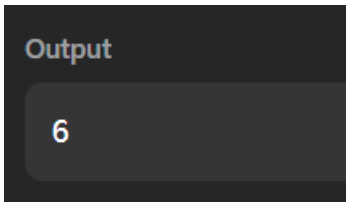
WEEK 5

TASK 5.1 : MAXIMUM SUBARRAY

PROGRAM:

```
class Solution {
    public int maxSubArray(int[] nums) {
        int currentSum = nums[0];
        int maxSum = nums[0];
        for (int i = 1; i < nums.length; i++) {
            currentSum = Math.max(nums[i], currentSum + nums[i]);
            maxSum = Math.max(maxSum, currentSum);
        }
        return maxSum;
    }
}
```

OUTPUT:



TASK 5.2: THE BIRTHDAY BAR

PROGRAM:

```
import java.util.*;
public class Solution {
    public static int birthday(List<Integer> s, int d, int m) {
        int count = 0;
        for (int i = 0; i <= s.size() - m; i++) {
            int sum = 0;
            for (int j = i; j < i + m; j++) {
                sum += s.get(j);
            }
            if (sum == d) {
                count++;
            }
        }
        return count;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        List<Integer> s = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            s.add(sc.nextInt());
        }
        int d = sc.nextInt();
        int m = sc.nextInt();
        System.out.println(birthday(s, d, m));
        sc.close();
    }
}
```

OUTPUT:



TASK 5.3: MAXSUBARRAY

PROGRAM:

```
import java.io.*;
import java.util.*;
class Result {
    public static List<Integer> maxSubarray(List<Integer> arr) {
        int current = arr.get(0);
        int maxSubarray = arr.get(0);
        int maxSubsequence = 0;
        int largestNegative = arr.get(0);
        for (int num : arr) {
            if (num != arr.get(0)) {
                current = Math.max(num, current + num);
                maxSubarray = Math.max(maxSubarray, current);
            }
            if (num > 0) {
                maxSubsequence += num;
            }
            if (num > largestNegative) {
                largestNegative = num;
            }
        }
        if (maxSubsequence == 0) {
            maxSubsequence = largestNegative;
        }
        List<Integer> result = new ArrayList<>();
        result.add(maxSubarray);
        result.add(maxSubsequence);
        return result;
    }
```

```
}
public class Solution {
    public static void main(String[] args) throws Exception {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        int t = Integer.parseInt(br.readLine().trim());
        while (t-- > 0) {
            int n = Integer.parseInt(br.readLine().trim());
            List<Integer> arr = new ArrayList<>();
            while (arr.size() < n) {
                String[] parts = br.readLine().trim().split("\\s+");
                for (String s : parts) {
                    if (!s.isEmpty()) {
                        arr.add(Integer.parseInt(s));
                    }
                }
            }
            List<Integer> result = Result.maxSubarray(arr);
            System.out.println(result.get(0) + " " + result.get(1));
        }
    }
}
```

OUTPUT:

Expected Output

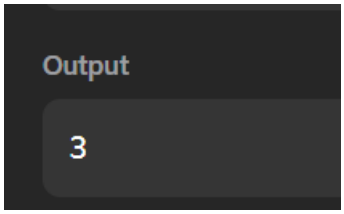
```
1 1
-1 -1
1 1
6 6
-10 -10
5 6
```

TASK 5.4:MAXIMUM SUB CIRCULAR SUBARRAY

PROBLEM:

```
class Solution {
    public int maxSubarraySumCircular(int[] nums) {
        int total = 0;
        int currentMax = 0;
        int maxSum = nums[0];
        int currentMin = 0;
        int minSum = nums[0];
        for (int num : nums) {
            currentMax = Math.max(num, currentMax + num);
            maxSum = Math.max(maxSum, currentMax);
            currentMin = Math.min(num, currentMin + num);
            minSum = Math.min(minSum, currentMin);
            total += num;
        }
        if (maxSum < 0) {
            return maxSum;
        }
        return Math.max(maxSum, total - minSum);
    }
}
```

OUTPUT:

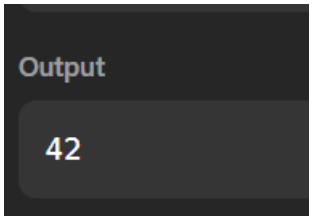


TASK 5.5:STRING TO INTEGER

PROGRAM:

```
class Solution {
    public int myAtoi(String s) {
        int i = 0;
        int n = s.length();
        while (i < n && s.charAt(i) == ' ') {
            i++;
        }
        int sign = 1;
        if (i < n && (s.charAt(i) == '+' || s.charAt(i) == '-')) {
            if (s.charAt(i) == '-') {
                sign = -1;
            }
            i++;
        }
        long num = 0;
        while (i < n && Character.isDigit(s.charAt(i))) {
            num = num * 10 + (s.charAt(i) - '0');
            if (sign == 1 && num > Integer.MAX_VALUE) {
                return Integer.MAX_VALUE;
            }
            if (sign == -1 && -num < Integer.MIN_VALUE) {
                return Integer.MIN_VALUE;
            }
            i++;
        }
        return (int)(sign * num);
    }
}
```

OUTPUT:



TASK 5.6 : ALTERNATING CHARACTERS

PROGRAM:

```
import java.io.*;
import java.util.*;
public class Solution {
    public static int alternatingCharacters(String s) {
        int deletions = 0;
        for (int i = 1; i < s.length(); i++) {
            if (s.charAt(i) == s.charAt(i - 1)) {
                deletions++;
            }
        }
        return deletions;
    }
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(
                new InputStreamReader(System.in)
        );
        int q = Integer.parseInt(br.readLine().trim());
        for (int i = 0; i < q; i++) {
            String s = br.readLine().trim();
            int result = alternatingCharacters(s);
            System.out.println(result);
        }
        br.close();
    }
}
```

OUTPUT:

Expected Output

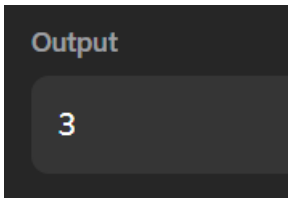
```
3
4
0
0
4
```

TASK 5.7 :LONGEST SUBSTRING

PROGRAM:

```
class Solution {
    public int lengthOfLongestSubstring(String s) {
        int left = 0;
        int maxLength = 0;
        HashSet<Character> set = new HashSet<>();
        for (int right = 0; right < s.length(); right++) {
            while (set.contains(s.charAt(right))) {
                set.remove(s.charAt(left));
                left++;
            }
            set.add(s.charAt(right));
            maxLength = Math.max(maxLength, right - left + 1);
        }
        return maxLength;
    }
}
```

OUTPUT:



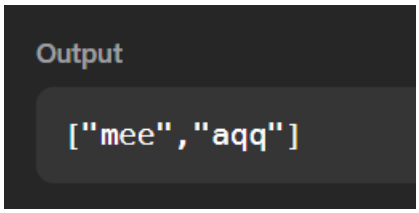
TASK 5.8:FIND AND REPLACE PATTERN

PROGRAM:

```
import java.util.*;
class Solution {
    public List<String> findAndReplacePattern(String[] words, String pattern) {
        List<String> result = new ArrayList<>();
        for (String word : words) {
            if (matches(word, pattern)) {
                result.add(word);
            }
        }
        return result;
    }
    private boolean matches(String word, String pattern) {
        Map<Character, Character> map = new HashMap<>();
        Map<Character, Character> reverseMap = new HashMap<>();
        for (int i = 0; i < word.length(); i++) {
            char w = word.charAt(i);
            char p = pattern.charAt(i);
            if (map.containsKey(p) && map.get(p) != w) {
                return false;
            }
            if (reverseMap.containsKey(w) && reverseMap.get(w) != p) {
                return false;
            }
            map.put(p, w);
            reverseMap.put(w, p);
        }
        return true;
    }
```

}

OUTPUT:

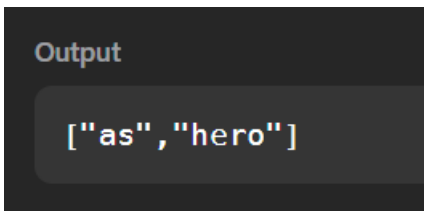


TASK 5.9: STRING MATCHING IN AN ARRAY

PROGRAM:

```
import java.util.*;
class Solution {
    public List<String> stringMatching(String[] words) {
        List<String> result = new ArrayList<>();
        for (int i = 0; i < words.length; i++) {
            for (int j = 0; j < words.length; j++) {
                if (i != j && words[j].contains(words[i])) {
                    result.add(words[i]);
                    break;
                }
            }
        }
        return result;
    }
}
```

OUTPUT:



TASK 5.10 :NAÏVE ALGORITHM

PROGRAM:

```
import java.util.ArrayList;
public class GFG {
    static ArrayList<Integer> search (String pat, String txt) {
        int m = pat.length();
        int n = txt.length();
        ArrayList<Integer> ans = new ArrayList<Integer>();
        for (int i = 0; i <= n - m; i++) {
            int j;
            for (j = 0; j < m; j++) {
                if (txt.charAt(i + j) != pat.charAt(j)) {
                    break;
                }
            }
            if (j == m) {
                ans.add(i);
            }
        }
        return ans;
    }
    public static void main(String[] args) {
        String txt = "aabaacaadaabaaba";
        String pat = "aaba";
        ArrayList<Integer> res = search(pat, txt);
        for (int i = 0; i < res.size(); i++) {
            System.out.print(res.get(i));
            if (i < res.size() - 1) {
                System.out.print(" ");
            }
```

```
        }
    }
}
```

OUTPUT:

Output

```
0 9 12
```

WEEK 6

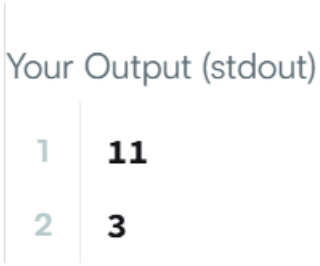
TASK 6.1: STRING SIMILARITY

PROGRAM:

```
import java.io.*;
import java.util.*;
public class Solution {
    public static long stringSimilarity(String s) {
        int n = s.length();
        int[] z = new int[n];
        int left = 0;
        int right = 0;
        long sum = n;
        for (int i = 1; i < n; i++) {
            if (i <= right) {
                z[i] = Math.min(right - i + 1, z [i - left]);
            }
            while (i + z[i] < n &&
                   s.charAt(z[i]) == s.charAt(i + z[i])) {
                z[i]++;
            }
            if (i + z[i] - 1 > right) {
                left = i;
                right = i + z[i] - 1;
            }
            sum += z[i];
        }
        return sum;
    }
    public static void main(String[] args) throws Exception {
      BufferedReader br =
```

```
        new BufferedReader(new InputStreamReader(System.in));
        int t = Integer.parseInt(br.readLine().trim());
        while (t-- > 0) {
            String s = br.readLine().trim();
            System.out.println(stringSimilarity(s));
        }
    }
}
```

OUTPUT:



TASK 6.2:REPEATED SUBSTRING

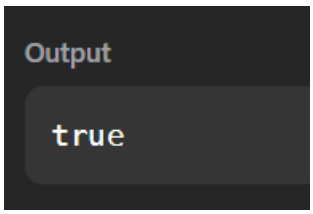
PROGRAM:

```
class Solution {
    public boolean repeatedSubstringPattern(String s) {

        String temp = s + s;

        return temp.substring(1, temp.length() - 1).contains(s);
    }
}
```

OUTPUT:



TASK 6.3:TWO STRINGS

PROGRAM:

```
import java.io.*;
import java.util.*;
public class Solution {
    public static String twoStrings(String s1, String s2) {
        boolean[] seen = new boolean[26];
        for (char c : s1.toCharArray()) {
            seen[c - 'a'] = true;
        }
        for (char c : s2.toCharArray()) {
            if (seen[c - 'a']) {
                return "YES";
            }
        }
        return "NO";
    }
    public static void main(String[] args) throws Exception {
        BufferedReader br =
            new BufferedReader(new InputStreamReader(System.in));
        int t = Integer.parseInt(br.readLine().trim());
        while (t-- > 0) {
            String s1 = br.readLine().trim();
            String s2 = br.readLine().trim();

            System.out.println(twoStrings(s1, s2));
        }
    }
}
```

OUTPUT:

Expected Output

```
1  YES
2  NO
```

TASK 6.4:ROTATE STRING

PROGRAM:

```
class Solution {
    public boolean rotateString(String s, String goal) {

        if (s.length() != goal.length()) {
            return false;
        }

        return (s + s).contains(goal);
    }
}
```

OUTPUT:

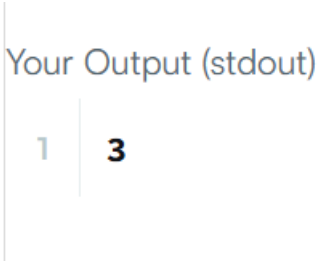


TASK 6.5:MARS EXPLORATION

PROGRAM:

```
import java.util.*;
public class Solution {
    public static int marsExploration(String s) {
        int count = 0;
        String message = "SOS";
        for (int i = 0; i < s.length(); i++) {
            if (s.charAt(i) != message.charAt(i % 3)) {
                count++;
            }
        }
        return count;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        int result = marsExploration(s);
        System.out.println(result);
        sc.close();
    }
}
```

OUTPUT:



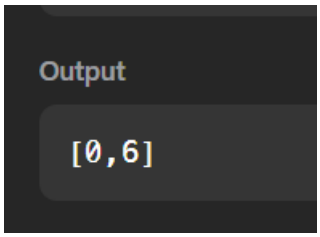
TASK 6.6: FIND ALL ANAGRAM

PROGRAM:

```
import java.util.*;
class Solution {
    public List<Integer> findAnagrams(String s, String p) {
        List<Integer> result = new ArrayList<>();
        if (s.length() < p.length()) {
            return result;
        }
        int[] count = new int[26];
        for (char c : p.toCharArray()) {
            count[c - 'a']++;
        }
        int left = 0;
        int right = 0;
        int required = p.length();
        while (right < s.length()) {
            char c = s.charAt(right);
            if (count[c - 'a'] > 0) {
                required--;
            }
            count[c - 'a']--;
            right++;
            if (right - left == p.length()) {
                if (required == 0) {
                    result.add(left);
                }
                char leftChar = s.charAt(left);
                count[leftChar - 'a']++;
```

```
            if (count[leftChar - 'a'] > 0) {
                required++;
            }
            left++;
        }
    }
    return result;
  }
}
```

OUTPUT:



TASK 6.7:PALINDROME INDEX

PROGRAM:

```
import java.io.*;
import java.util.*;
public class Solution {
    public static int palindromeIndex(String s) {
        int left = 0;
        int right = s.length() - 1;
        while (left < right) {
            if (s.charAt(left) != s.charAt(right)) {
                if (isPalindrome(s, left + 1, right)) {
                    return left;
                }
                if (isPalindrome(s, left, right - 1)) {
                    return right;
                }
                return -1;
            }
            left++;
            right--;
        }
        return -1;
    }
    private static boolean isPalindrome(String s, int left, int right) {
        while (left < right) {
            if (s.charAt(left) != s. charAt(right)) {
                return false;
            }
            left++;
            right--;
```

```
        }
        return true;
    }
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        int q = sc.nextInt();
        while (q-- > 0) {
            String s = sc.next();
            System.out.println(palindromeIndex(s));
        }
        sc.close();
    }
}
```

OUTPUT:

Expected Output

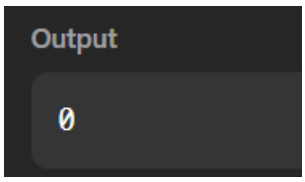
```
3
0
-1
```

TASK 6.8:FIND THE INDEX

PROGRAM:

```
class Solution {
    public int strStr(String haystack, String needle) {
        int n = haystack.length();
        int m = needle.length();
        if (m > n) {
            return -1;
        }
        for (int i = 0; i <= n - m; i++) {
            int j = 0;
            while (j < m && haystack.charAt(i + j) == needle.charAt(j)) {
                j++;
            }
            if (j == m) {
                return i;
            }
        }
        return -1;
    }
}
```

OUTPUT:



TASK 6.9:LONGEST PALINDROMIC SUBSTRING

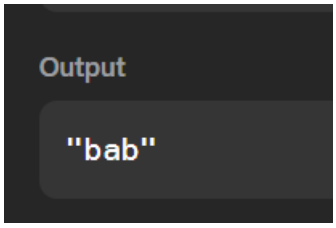
PROGRAM:

```
class Solution {
    public String longestPalindrome(String s) {
        int start = 0;
        int maxLength = 1;
        for (int i = 0; i < s.length(); i++) {
            int left = i;
            int right = i;
            while (left >= 0 && right < s.length()
                    && s.charAt(left) == s.charAt(right)) {
                if (right - left + 1 > maxLength) {
                    start = left;
                    maxLength = right - left + 1;
                }
                left--;
                right++;
            }

            left = i;
            right = i + 1;
            while (left >= 0 && right < s.length()
                    && s.charAt(left) == s.charAt(right)) {
                if (right - left + 1 > maxLength) {
                    start = left;
                    maxLength = right - left + 1;
                }
                left--;
                right++;
            }
```

```
        }
        return s.substring(start, start + maxLength);
    }
}
```

OUTPUT:



TASK 6.10:CIRCULAR PALINDROMES

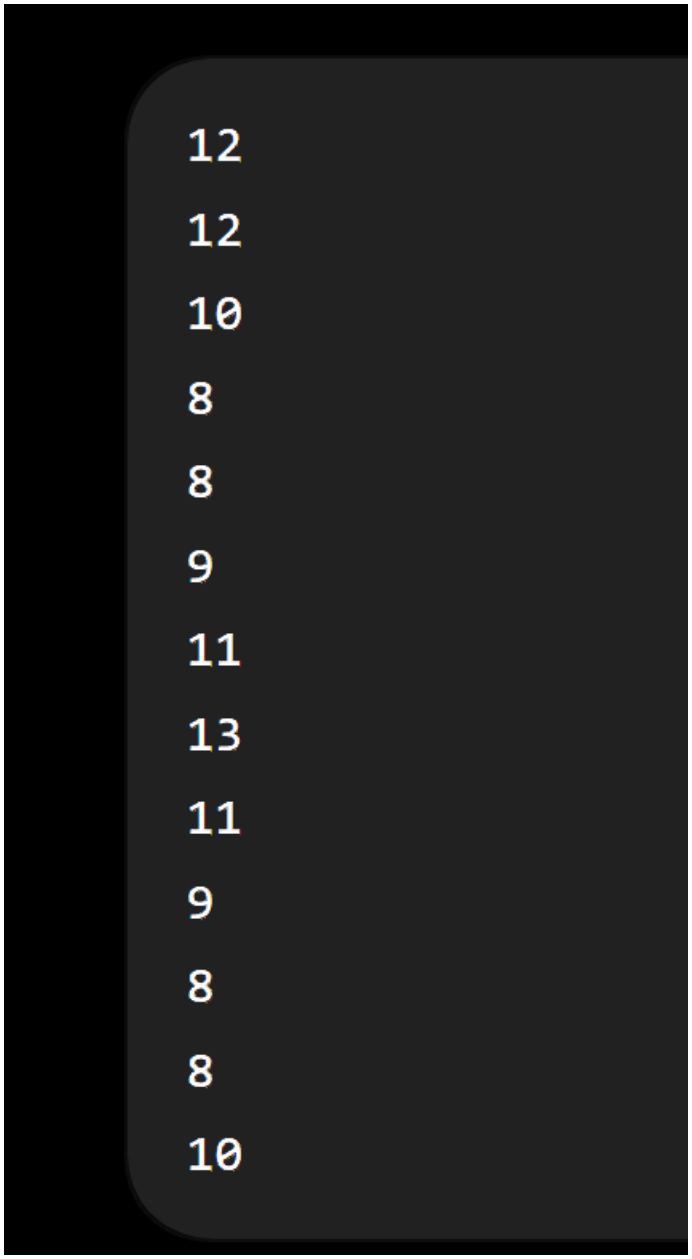
PROGRAM:

```
import java.io.*;
import java.util.*;
public class Solution {
    static int[] manacher(String s) {
        int n = s.length();
        StringBuilder t = new StringBuilder();
        t.append('#');
        for (int i = 0; i < n; i++) {
            t.append(s.charAt(i));
            t.append('#');
        }
        int m = t.length();
        int[] p = new int[m];
        int center = 0;
        int right = 0;
        for (int i = 0; i < m; i++) {
            int mirror = 2 * center - i;
            if (i < right) {
                p[i] = Math.min(right - i, p[mirror]);
            }
            while (i + p[i] + 1 < m &&
                   i - p[i] - 1 >= 0 &&
                   t.charAt(i + p[i] + 1)
                   == t.charAt(i - p[i] - 1)) {
                p[i]++;
            }
            if (i + p[i] > right) {
                center = i;
```

```
            right = i + p[i];
        }
    }
    return p;
}
static int longestPalindrome(String s) {
    if (s.length() == 0) {
        return 0;
    }
    int[] p = manacher(s);
    int maximum = 0;
    for (int i = 0; i < p.length; i++) {
        if (p[i] > maximum) {
            maximum = p[i];
        }
    }
    return maximum;
}
public static List<Integer> circularPalindromes(String s) {
    int n = s.length();
    List<Integer> result = new ArrayList<Integer>();
    String current = s;
    for (int rotation = 0; rotation < n; rotation++) {
        int maximum = longestPalindrome(current);
        result.add(maximum);
        current = current.substring(1)
                + current.charAt(0);
    }
    return result;
}
```

```
    public static void main(String[] args) throws Exception {
        BufferedReader br =
                new BufferedReader(
                        new InputStreamReader(System.in)
                );
        int n = Integer.parseInt(br.readLine().trim());
        String s = br.readLine().trim();
        List<Integer> answer =
                circularPalindromes(s);
        StringBuilder output =
                new StringBuilder();
        for (int i = 0; i < answer.size(); i++) {
            output.append(answer.get(i));
            if (i < answer.size() - 1) {
                output.append('\n');
            }
        }
        System.out.print(output.toString());
    }
}
```

OUTPUT:


12
12
10
8
8
9
11
13
11
9
8
8
10

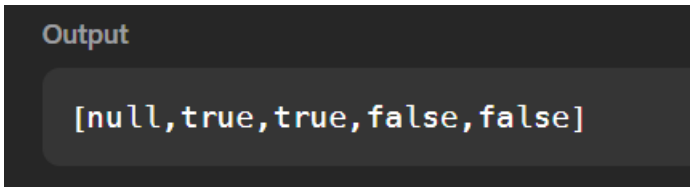
TASK 7.1: DESIGN PARKING SYSTEM

PROGRAM:

```
class ParkingSystem {
    int big;
    int medium;
    int small;
    public ParkingSystem(int big, int medium, int small) {
        this.big = big;
        this.medium = medium;
        this.small = small;
    }
    public boolean addCar(int carType) {
        if (carType == 1) {
            if (big > 0) {
                big--;
                return true;
            }
        }
        else if (carType == 2) {
            if (medium > 0) {
                medium--;
                return true;
            }
        }
        else if (carType == 3) {
            if (small > 0) {
                small--;
                return true;
            }
        }
```

```
        return false;
    }
}
```

OUTPUT:



TASK 7.2 : JAVA INSTANCES OF A KEYWORD

PROGRAM:

```
import java.util.*;
class Student{}
class Rockstar{}
class Hacker{}
public class InstanceOFTutorial{

    static String count(ArrayList mylist){
        int a = 0, b = 0, c = 0;

        for(int i = 0; i < mylist.size(); i++){
            Object element = mylist.get(i);

            if(element instanceof Student)
                a++;

            if(element instanceof Rockstar)
                b++;

            if(element instanceof Hacker)
                c++;
        }

        String ret = Integer.toString(a) + " "
                + Integer.toString(b) + " "
                + Integer.toString(c);

        return ret;
    }
```

```
    public static void main(String []args){
        ArrayList mylist = new ArrayList();
        Scanner sc = new Scanner(System.in);

        int t = sc.nextInt();

        for(int i = 0; i < t; i++){
            String s = sc.next();

            if(s.equals("Student"))
                mylist.add(new Student());

            if(s.equals("Rockstar"))
                mylist.add(new Rockstar());

            if(s.equals("Hacker"))
                mylist.add(new Hacker());
        }

        System.out.println(count(mylist));
    }
}import java.util.*;

class Student{}
class Rockstar{}
class Hacker{}

public class InstanceOFTutorial{
```

```
static String count(ArrayList mylist){
    int a = 0, b = 0, c = 0;

    for(int i = 0; i < mylist.size(); i++){
        Object element = mylist.get(i);

        if(element instanceof Student)
            a++;

        if(element instanceof Rockstar)
            b++;

        if(element instanceof Hacker)
            c++;
    }

    String ret = Integer.toString(a) + " "
            + Integer.toString(b) + " "
            + Integer.toString(c);

    return ret;
}

public static void main(String []args){
    ArrayList mylist = new ArrayList();
    Scanner sc = new Scanner(System.in);

    int t = sc.nextInt();

    for(int i = 0; i < t; i++){
```

```
            String s = sc.next();

            if(s.equals("Student"))
                mylist.add(new Student());

            if(s.equals("Rockstar"))
                mylist.add(new Rockstar());

            if(s.equals("Hacker"))
                mylist.add(new Hacker());
        }

        System.out.println(count(mylist));
    }
}
```

OUTPUT:

Expected Output

```
3 1 1
```

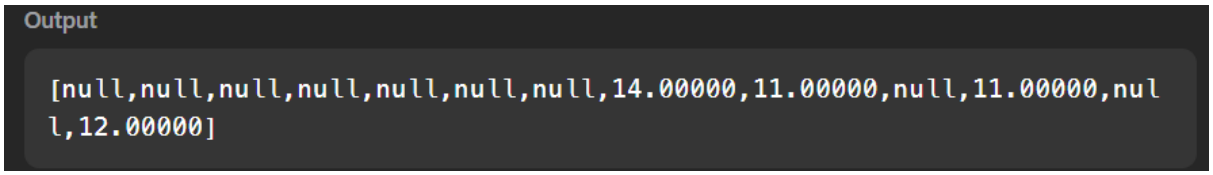
TASK 7.3 : DESIGN UNDERGROUND SYSTEM

PROGRAM:

```
import java.util.*;
class UndergroundSystem {
    HashMap<Integer, CheckIn> checkIns;
    HashMap<String, Route> routes;
    class CheckIn {
        String station;
        int time;
        CheckIn(String station, int time) {
            this.station = station;
            this.time = time;
        }
    }
    class Route {
        int totalTime;
        int count;
        Route(int totalTime, int count) {
            this.totalTime = totalTime;
            this.count = count;
        }
    }
    public UndergroundSystem() {
        checkIns = new HashMap<>();
        routes = new HashMap<>();
    }
    public void checkIn(int id, String stationName, int t) {
        checkIns.put(id, new CheckIn(stationName, t));
    }
```

```
    public void checkOut(int id, String stationName, int t) {
        CheckIn c = checkIns.get(id);
        String key = c.station + "#" + stationName;
        int travelTime = t - c.time;
        if (routes.containsKey(key)) {
            Route r = routes.get(key);
            r.totalTime += travelTime;
            r.count++;
        } else {
            routes.put(key, new Route(travelTime, 1));
        }
        checkIns.remove(id);
    }
    public double getAverageTime(String startStation, String endStation) {
        String key = startStation + "#" + endStation;
        Route r = routes.get(key);
        return (double) r.totalTime / r.count;
    }
}
```

OUTPUT:



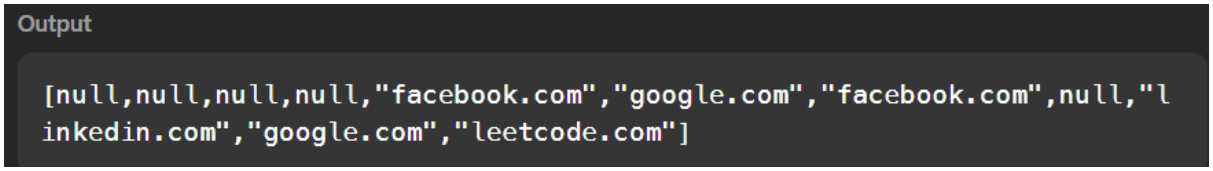
TASK 7.4 : DESIGN BROWSER HISTORY

PROGRAM:

```
class BrowserHistory {
    class Node {
        String url;
        Node prev;
        Node next;
        Node(String url) {
            this.url = url;
        }
    }
    Node current;
    public BrowserHistory(String homepage) {
        current = new Node(homepage);
    }
    public void visit(String url) {
        Node newNode = new Node(url);
        current.next = newNode;
        newNode.prev = current;
        current = newNode;
    }
    public String back(int steps) {
        while (steps > 0 && current.prev != null) {
            current = current.prev;
            steps--;
        }
        return current.url;
    }
    public String forward(int steps) {
        while (steps > 0 && current.next != null) {
```

```
            current = current.next;
            steps--;
        }
        return current.url;
    }
}
```

OUTPUT:



TASK 7.5 : HASHSET

PROGRAM:

```
class MyHashSet {
    boolean[] set;
    public MyHashSet() {
        set = new boolean[1000001];
    }
    public void add(int key) {
        set[key] = true;
    }
    public void remove(int key) {
        set[key] = false;
    }
    public boolean contains(int key) {
        return set[key];
    }
}
```

OUTPUT:

Output

```
[null,null,null,null,null,false,null,null,null,null,false,null,null,nul
l,null,false,null,null,null,null,false]
```

TASK 7.6 : GRADING STUDENTS

PROGRAM:

```
import java.io.*;
import java.util.*;
import java.util.stream.*;
import static java.util.stream.Collectors.toList;
class Result {
    public static List<Integer> gradingStudents(List<Integer> grades) {
        for (int i = 0; i < grades.size(); i++) {
            int grade = grades.get(i);
            if (grade >= 38) {
                int next = ((grade / 5) + 1) * 5;
                if (next - grade < 3) {
                    grades.set(i, next);
                }
            }
        }
        return grades;
    }
}
public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader =
                new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter =
                new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));
        int gradesCount =
                Integer.parseInt(bufferedReader.readLine().trim());
        List<Integer> grades = IntStream.range(0, gradesCount)
                .mapToObj(i -> {
```

```
                try {
                    return bufferedReader.readLine().replaceAll("\\s+$", "");
                } catch (IOException ex) {
                    throw new RuntimeException(ex);
                }
            })
            .map(Integer::parseInt)
            .collect(toList());
        List<Integer> result = Result.gradingStudents(grades);
        bufferedWriter.write(
            result.stream()
                .map(Object::toString)
                .collect(Collectors.joining("\n"))
                + "\n"
        );
        bufferedReader.close();
        bufferedWriter.close();
    }
}
```

OUTPUT:

Your Output (stdout)

```
75
67
40
33
```

TASK 7.7 : JAVA INHERITANCE 1

PROGRAM:

```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;
class Animal {
    void walk() {
        System.out.println("I am walking");
    }
}
class Bird extends Animal {
    void fly() {
        System.out.println("I am flying");
    }
    void sing() {
        System.out.println("I am singing");
    }
}
public class Solution {
    public static void main(String[] args) {
        Bird bird = new Bird();
        bird.walk();
        bird.fly();
        bird.sing();
    }
}
```

OUTPUT:

Your Output (stdout)

```
I am walking
I am flying
I am singing
```

TASK 7. 8: JAVA INHERITANCE 2

PROGRAM:

```
import java.io.*;
import java.util.*;
class Arithmetic {
    int add(int a, int b) {
        return a + b;
    }
}
class Adder extends Arithmetic {
}
public class Solution {
    public static void main(String[] args) {
        Adder adder = new Adder();
        System.out.println("My superclass is: "
                + adder.getClass().getSuperclass().getName());
        System.out.println(adder.add(42, 13) + " "
                + adder.add(10, 10) + " "
                + adder.add(5, 15));
    }
}
```

OUTPUT:

Expected Output

```
My superclass is: Arithmetic
42 13 20
```

TASK 7.9 : JAVA GENERICS

PROGRAM:

```
import java.io.*;
import java.util.*;
public class Solution {
    public static <T> void printArray(T[] array) {
        for (T element : array) {
            System.out.println(element);
        }
    }
    public static void main(String[] args) {
        Integer[] intArray = {1, 2, 3};
        String[] stringArray = {"Hello", "World"};
        printArray(intArray);
        printArray(stringArray);
    }
}
```

OUTPUT:



TASK 7.10: THRONE INHERITANCE

PROGRAM:

```
import java.util.*;
class ThroneInheritance {
    String king;
    Map<String, List<String>> children;
    Set<String> dead;
    public ThroneInheritance(String kingName) {
        king = kingName;
        children = new HashMap<>();
        dead = new HashSet<>();
    }
    public void birth(String parentName, String childName) {
        children.putIfAbsent(parentName, new ArrayList<>());
        children.get(parentName).add(childName);
    }
    public void death(String name) {
        dead.add(name);
    }
    public List<String> getInheritanceOrder() {
        List<String> result = new ArrayList<>();
        dfs(king, result);
        return result;
    }
    private void dfs(String person, List<String> result) {
        if (!dead.contains(person)) {
            result.add(person);
        }
        if (children.containsKey(person)) {
            for (String child : children.get(person)) {
```

```
            dfs(child, result);
        }
    }
  }
}
```

OUTPUT:

Output


```
[null,null,null,null,null,null,null,["king","andy","matthew","bob","a
x","asha","catherine"],null,["king","andy","matthew","alex","asha","cat
herine"]]
```

WEEK 8

1. import java.util.*;

```
class Sports{

   String getName(){
      return "Generic Sports";
   }

   void getNumberOfTeamMembers(){
      System.out.println( "Each team has n players in " + getName() );
   }
}

class Soccer extends Sports{
   @Override
   String getName(){
      return "Soccer Class";
   }


   void getNumberOfTeamMembers() {
      System.out.println("Each team has 11 players in " + getName());
   }

}

public class Solution{

   public static void main(String []args){
      Sports c1 = new Sports();
```

```
        Soccer c2 = new Soccer();
        System.out.println(c1.getName());
        c1.getNumberOfTeamMembers();
        System.out.println(c2.getName());
        c2.getNumberOfTeamMembers();
    }
}
```

Compiler Message

Success

Expected Output Download

```
Generic Sports
Each team has n players in Generic Sports
Soccer Class
Each team has 11 players in Soccer Class
```

2.
```
import java.util.*;
import java.io.*;

class BiCycle{
    String define_me(){
        return "a vehicle with pedals.";
    }
}

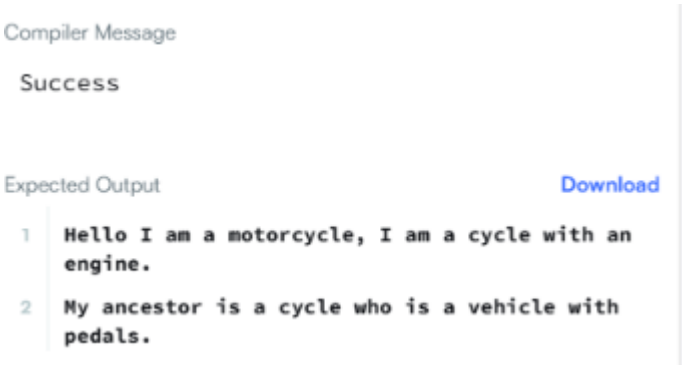
class MotorCycle extends BiCycle{
    String define_me(){
        return "a cycle with an engine.";
    }

    MotorCycle(){
        System.out.println("Hello I am a motorcycle, I am "+ define_me());
```

```
String temp = super.define_me();

        System.out.println("My ancestor is a cycle who is "+ temp );
    }

}
class Solution{
    public static void main(String []args){
        MotorCycle M=new MotorCycle();
    }
}
```



3.
```
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;
import java.lang.reflect.*;
class Singleton {
    // 1. Static reference to the single instance of the class
    private static Singleton instance;

    // 2. Public String instance variable
```

```
    public String str;

    // 3. Private constructor to prevent direct instantiation
    private Singleton() {
    }

    // 4. Static method to return the single instance
    public static Singleton getSingleInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}

public class Main {

 public static void main(String args[])throws Exception{

    Scanner sc=new Scanner(System.in);
    Singleton s1 = Singleton.getSingleInstance(); //retrive the single instance
    Singleton s2=Singleton.getSingleInstance();
    assert(s1==s2);

    //verify that the constructor is private
    Class c=s1.getClass();
    Constructor[] allConstructors = c.getDeclaredConstructors();
    assert allConstructors.length==1;
    for (Constructor ctor : allConstructors)
    {
```

```
      if(ctor.getModifiers()!=2 || !ctor.toString().equals("private Singleton()")) //The constructor must be private
      {
         System.out.println("Wrong class!");
      }
   }
   String str=sc.nextLine();
   s1.str=str;
   s2.str=str;
   System.out.println("Hello I am a singleton! Let me say "+str+" to you");

 }

}
```




```
4import java.util.ArrayList;
import java.io.*;
import java.util.*;
import java.text.*;
import java.math.*;
import java.util.regex.*;

enum Color {
   RED, GREEN
```

```
}

abstract class Tree {

    private int value;
    private Color color;
    private int depth;

    public Tree(int value, Color color, int depth) {
        this.value = value;
        this.color = color;
        this.depth = depth;
    }

    public int getValue() {
        return value;
    }

    public Color getColor() {
        return color;
    }

    public int getDepth() {
        return depth;
    }

    public abstract void accept(TreeVis visitor);
}

class TreeNode extends Tree {
```

```
    private ArrayList<Tree> children = new ArrayList<>();

    public TreeNode(int value, Color color, int depth) {
        super(value, color, depth);
    }

    public void accept(TreeVis visitor) {
        visitor.visitNode(this);

        for (Tree child : children) {
            child.accept(visitor);
        }
    }

    public void addChildrenNode(Tree node) {
        children.add(node);
    }
}

class TreeLeaf extends Tree {

    public TreeLeaf(int value, Color color, int depth) {
        super(value, color, depth);
    }

    public void accept(TreeVis visitor) {
        visitor.visitLeaf(this);
    }
}

abstract class TreeVis {
```

```
    public abstract int getResult();

    public abstract void visitNode(TreeNode node);

    public abstract void visitLeaf(TreeLeaf leaf);

}

class SumInLeavesVisitor extends TreeVis {
    private int sum = 0;

    public int getResult() {
        return sum;
    }

    public void visitNode(TreeNode node) {
        // Implementation for non-leaf node if needed
    }

    public void visitLeaf(TreeLeaf leaf) {
        sum += leaf.getValue();
    }
}

class ProductRedNodesVisitor extends TreeVis {
    private long product = 1;
    private boolean hasRedNode = false;
    private final long MOD = 1_000_000_007;

    public int getResult() {
        return hasRedNode ? (int) product : 0;
```

```
    }

    public void visitNode(TreeNode node) {
        if (node.getColor() == Color.RED) {
            hasRedNode = true;
            product = (product * node.getValue()) % MOD;
        }
    }

    public void visitLeaf(TreeLeaf leaf) {
        if (leaf.getColor() == Color.RED) {
            hasRedNode = true;
            product = (product * leaf.getValue()) % MOD;
        }
    }
}

class FancyVisitor extends TreeVis {
    private int nonLeafEvenDepthSum = 0;
    private int greenLeafSum = 0;

    public int getResult() {
        return Math.abs(nonLeafEvenDepthSum - greenLeafSum);
    }

    public void visitNode(TreeNode node) {
        if (node.getDepth() % 2 == 0) {
            nonLeafEvenDepthSum += node.getValue();
        }
    }
```

```
    public void visitLeaf(TreeLeaf leaf) {
        if (leaf.getColor() == Color.GREEN) {
            greenLeafSum += leaf.getValue();
        }
    }
}

public class Solution {

    // Helper structures to build the tree from adjacency lists
    private static int[] values;
    private static Color[] colors;
    private static List<List<Integer>> adj;
    private static boolean[] visited;

    public static Tree solve() {
        Scanner scanner = new Scanner(System.in);
        int n = scanner.nextInt();

        values = new int[n + 1];
        for (int i = 1; i <= n; i++) {
            values[i] = scanner.nextInt();
        }

        colors = new Color[n + 1];
        for (int i = 1; i <= n; i++) {
            colors[i] = scanner.nextInt() == 0 ? Color.RED : Color.GREEN;
        }

        adj = new ArrayList<>();
        for (int i = 0; i <= n; i++) {
```

```
        adj.add(new ArrayList<>());
    }

    for (int i = 0; i < n - 1; i++) {
        int u = scanner.nextInt();
        int v = scanner.nextInt();
        adj.get(u).add(v);
        adj.get(v).add(u);
    }
    scanner.close();

    if (n == 1) {
        return new TreeLeaf(values[1], colors[1], 0);
    }

    visited = new boolean[n + 1];
    return buildTree(1, 0);
}

private static Tree buildTree(int u, int depth) {
    visited[u] = true;

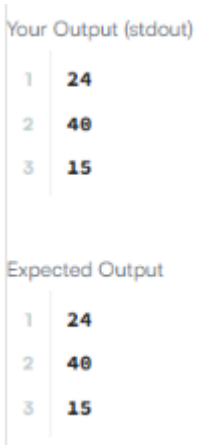
    // Check if node u is a leaf (has no unvisited neighbors except its parent)
    boolean isLeaf = true;
    for (int v : adj.get(u)) {
        if (!visited[v]) {
            isLeaf = false;
            break;
        }
    }
```

```
        if (isLeaf) {
            return new TreeLeaf(values[u], colors[u], depth);
        } else {
            TreeNode node = new TreeNode(values[u], colors[u], depth);
            for (int v : adj.get(u)) {
                if (!visited[v]) {
                    node.addChildrenNode(buildTree(v, depth + 1));
                }
            }
            return node;
        }
    }

    public static void main(String[] args) {
        Tree root = solve();
        SumInLeavesVisitor vis1 = new SumInLeavesVisitor();
        ProductRedNodesVisitor vis2 = new ProductRedNodesVisitor();
        FancyVisitor vis3 = new FancyVisitor();

        root.accept(vis1);
        root.accept(vis2);
        root.accept(vis3);

        System.out.println(vis1.getResult());
        System.out.println(vis2.getResult());
        System.out.println(vis3.getResult());
    }
}
```



5. import java.util.*;

```
public class Main{


  static Iterator func(ArrayList mylist){
    Iterator it=mylist.iterator();
    while(it.hasNext()){
      Object element = it.next();
        if (element instanceof String && element.equals("###"))


        break;
     }
    return it;


  }
  @SuppressWarnings({ "unchecked" })
  public static void main(String []args){
    ArrayList mylist = new ArrayList();
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    int m = sc.nextInt();
    for(int i = 0;i<n;i++){
      mylist.add(sc.nextInt());
    }
```

```
    mylist.add("###");
    for(int i=0;i<m;i++){
      mylist.add(sc.next());
    }

    Iterator it=func(mylist);
    while(it.hasNext()){
      Object element = it.next();
      System.out.println((String)element);
    }
  }
}
```

Your Output (stdout)

```
1 hello
2 java
```

Expected Output

```
1 hello
2 java
```

6.
```
import java.util.*;
interface AdvancedArithmetic{
  int divisor_sum(int n);
}

class MyCalculator implements AdvancedArithmetic {

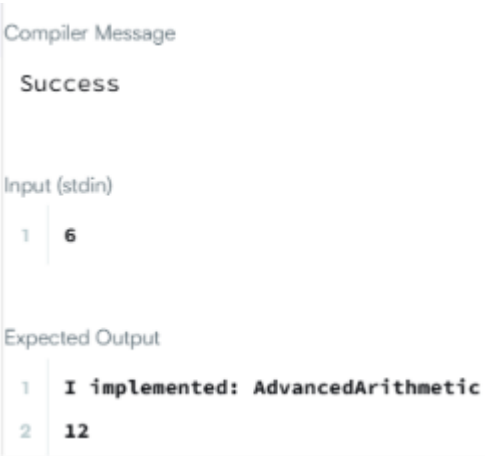
    @Override
    public int divisor_sum(int n) {
        int sum = 0;
```

```
        for (int i = 1; i <= n; i++) {
            if (n % i == 0) {
                sum += i;
            }
        }

        return sum;
    }
}

class Solution{
    public static void main(String []args){
        MyCalculator my_calculator = new MyCalculator();
        System.out.print("I implemented: ");
        ImplementedInterfaceNames(my_calculator);
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        System.out.print(my_calculator.divisor_sum(n) + "\n");
        sc.close();
    }
    /*
     *  ImplementedInterfaceNames method takes an object and prints the name of the interfaces it implemented
     */
    static void ImplementedInterfaceNames(Object o){
        Class[] theInterfaces = o.getClass().getInterfaces();
        for (int i = 0; i < theInterfaces.length; i++){
            String interfaceName = theInterfaces[i].getName();
            System.out.println(interfaceName);
        }
    }
```

}



7. import java.io.*;

import java.util.*;

import java.text.*;

import java.math.*;

import java.util.regex.*;

public class Solution {

public static void main(String[] args) {

Scanner s = new Scanner(System.in);

int t = s.nextInt();

String [] pair_left = new String[t];

String [] pair_right = new String[t];

for (int i = 0; i < t; i++) {

pair_left[i] = s.next();

pair_right[i] = s.next();

}

HashSet<String> set = new HashSet<>(t * 2);

StringBuilder sb = new StringBuilder();

```
        for (int i = 0; i < t; i++) {
            set.add(pair_left[i] + " " + pair_right[i]);
            sb.append(set.size()).append("\n");
        }

        System.out.print(sb);
    }
}
```

```
5
john tom
john mary
john tom
mary anna
mary anna
```

Your Output (stdout)

```
1
2
2
3
3
```

8.

```
import java.util.*;
import java.security.*;
interface Food {
    public String getType();
   }
   class Pizza implements Food {
    public String getType() {
    return "Someone ordered a Fast Food!";
    }
   }

   class Cake implements Food {

    public String getType() {
    return "Someone ordered a Dessert!";
```

```
    }
   }
   class FoodFactory {
      public Food getFood(String order) {
  if (order.equals("pizza")) {
         return new Pizza();
      } else {
         return new Cake();
      }

}//End of getFood method

   }//End of factory class

   public class Solution {

    public static void main(String args[]){
        Do_Not_Terminate.forbidExit();

      try{

         Scanner sc=new Scanner(System.in);
         //creating the factory
         FoodFactory foodFactory = new FoodFactory();

         //factory instantiates an object
         Food food = foodFactory.getFood(sc.nextLine());



         System.out.println("The factory returned "+food.getClass());
         System.out.println(food.getType());
```

```
    }
    catch (Do_Not_Terminate.ExitTrappedException e) {
        System.out.println("Unsuccessful Termination!!");
    }
  }

}
class Do_Not_Terminate {

    public static class ExitTrappedException extends SecurityException {

        private static final long serialVersionUID = 1L;
    }

    public static void forbidExit() {
        final SecurityManager securityManager = new SecurityManager() {
            @Override
            public void checkPermission(Permission permission) {
                if (permission.getName().contains("exitVM")) {
                    throw new ExitTrappedException();
                }
            }
        };
        System.setSecurityManager(securityManager);
    }
}
```

Compiler Message

Success

Input (stdin)

```
cake
```

Expected Output

```
The factory returned class Cake
Someone ordered a Dessert!
```

9.
```
import java.util.Scanner;
class MyCalculator {

   /*
    * Create the method long power(int, int) here.
    */

   public long power(int n, int p) throws Exception {

      if (n < 0 || p < 0) {
         throw new Exception("n or p should not be negative.");
      }

      if (n == 0 && p == 0) {
         throw new Exception("n and p should not be zero.");
      }

      long result = 1;

      for (int i = 0; i < p; i++) {
         result *= n;
      }

      return result;
   }
}

public class Solution {
   public static final MyCalculator my_calculator = new MyCalculator();
   public static final Scanner in = new Scanner(System.in);
```

```
    public static void main(String[] args) {
        while (in .hasNextInt()) {
            int n = in .nextInt();
            int p = in .nextInt();

            try {
                System.out.println(my_calculator.power(n, p));
            } catch (Exception e) {
                System.out.println(e);
            }
        }
    }
}
```

```
2 4
0 0
-1 -2
-1 3
```

Expected Output Downloac

```
243
16
java.lang.Exception: n and p should not be zero.
java.lang.Exception: n or p should not be negative.
java.lang.Exception: n or p should not be
```

10.
```
import java.util.*;

public class Solution {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        try {
            int a = sc.nextInt();
```

```
            int b = sc.nextInt();


            System.out.println(a / b);
        }
        catch (InputMismatchException e) {
            System.out.println(e.getClass().getName());
        }
        catch (ArithmeticException e) {
            System.out.println(e);
        }


        sc.close();
    }
}
```

Compiler Message

```
Success
```

Input (stdin)

```
10
3
```

Expected Output

```
3
```

WEEK 9

1.
```
import java.util.*;


class BankAccount {
    int balance = 0;

    void deposit(int amount) {
        balance = balance + amount;
    }

    void withdraw(int amount) {
        balance = balance - amount;
    }

    int getBalance() {
        return balance;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        sc.nextLine();

        BankAccount account = new BankAccount();

        for (int i = 0; i < n; i++) {
            String operation = sc.nextLine();
```

```
            String[] parts = operation.split(" ");

            String type = parts[0];
            int amount = Integer.parseInt(parts[1]);

            if (type.equals("Deposit")) {
                account.deposit(amount);
            } else if (type.equals("Withdraw")) {
                account.withdraw(amount);
            }
        }

        System.out.println(account.getBalance());
    }
}
```

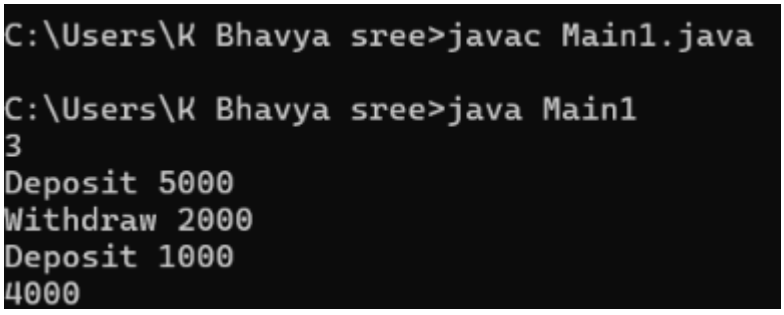
Input:

3

Deposit 5000

Withdraw 2000

Deposit 1000



2. import java.util.*;

```
class Driver {
    String name;
```

```
    Driver(String name) {
        this.name = name;
    }
}

class Rider {
    String name;

    Rider(String name) {
        this.name = name;
    }
}

class Vehicle {
    String number;

    Vehicle(String number) {
        this.number = number;
    }
}

// Parent class
abstract class Ride {
    abstract int calculateFare(int distance);
}

// Bike
class Bike extends Ride {
    int calculateFare(int distance) {
        return distance * 5;
```

```
    }
}

// Cab
class Cab extends Ride {
    int calculateFare(int distance) {
        return distance * 12;
    }
}

// Auto
class Auto extends Ride {
    int calculateFare(int distance) {
        return distance * 12;
    }
}

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        for (int i = 0; i < n; i++) {

            String type = sc.next();
            int distance = sc.nextInt();

            try {
                if (distance <= 0) {
```

```
                throw new Exception("Invalid distance");
            }

            Ride ride;

            if (type.equals("Bike")) {
                ride = new Bike();
            }
            else if (type.equals("Cab")) {
                ride = new Cab();
            }
            else if (type.equals("Auto")) {
                ride = new Auto();
            }
            else {
                throw new Exception("Invalid ride type");
            }

            System.out.println(ride.calculateFare(distance));

        } catch (Exception e) {
            System.out.println("Invalid booking");
        }
    }

    sc.close();
  }
}
```

Input:

3

Bike 10

Cab 15

Auto 8

```
C:\Users\K Bhavya sree>javac Main2.java

C:\Users\K Bhavya sree>java Main2
3
Bike 10
Cab 15
50
180
Auto 8
96
```

3. import java.util.Scanner;

```
class Vehicle {
    String vehicleNumber;
    double rentPerDay;

    Vehicle(String vehicleNumber, double rentPerDay) {
        this.vehicleNumber = vehicleNumber;
        this.rentPerDay = rentPerDay;
    }

    double calculateRent(int days) {
        return rentPerDay * days;
    }
}

class Car extends Vehicle {

    Car(String vehicleNumber, double rentPerDay) {
        super(vehicleNumber, rentPerDay);
    }
```

```
    @Override
    double calculateRent(int days) {
        return rentPerDay * days;
    }
}

class Bike extends Vehicle {

    Bike(String vehicleNumber, double rentPerDay) {
        super(vehicleNumber, rentPerDay);
    }

    @Override
    double calculateRent(int days) {
        return rentPerDay * days * 0.90;
    }
}

class Truck extends Vehicle {

    Truck(String vehicleNumber, double rentPerDay) {
        super(vehicleNumber, rentPerDay);
    }

    @Override
    double calculateRent(int days) {
        return rentPerDay * days * 1.20;
    }
}

public class Main3 {
```

```
public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    int N = sc.nextInt();

    for (int i = 0; i < N; i++) {

        int vehicleType = sc.nextInt();
        String vehicleNumber = sc.next();
        double rentPerDay = sc.nextDouble();
        int days = sc.nextInt();

        Vehicle vehicle;

        if (vehicleType == 1) {
            vehicle = new Car(vehicleNumber, rentPerDay);
        }
        else if (vehicleType == 2) {
            vehicle = new Bike(vehicleNumber, rentPerDay);
        }
        else {
            vehicle = new Truck(vehicleNumber, rentPerDay);
        }

        double totalRent = vehicle.calculateRent(days);

        System.out.printf("%s %.2f%n", vehicleNumber, totalRent);
    }
```

```
        sc.close();
    }
}
```

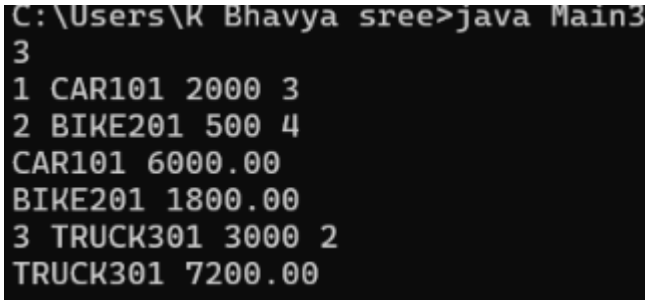
Input:

3

1 CAR101 2000 3

2 BIKE201 500 4

3 TRUCK301 3000 2



4.
```
import java.util.Scanner;


interface Payment {
    void pay(double amount);
}


class CreditCardPayment implements Payment {
    public void pay(double amount) {
        System.out.println("Credit Card payment: " + amount);
    }
}


class UPIPayment implements Payment {
    public void pay(double amount) {
        System.out.println("UPI payment: " + amount);
    }
}
```

```
class NetBankingPayment implements Payment {
    public void pay(double amount) {
        System.out.println("Net Banking payment: " + amount);
    }
}

abstract class PaymentProcessor {
    abstract double processPayment(Payment payment, double amount);
}

class OnlinePaymentProcessor extends PaymentProcessor {

    @Override
    double processPayment(Payment payment, double amount) {

        if (payment instanceof CreditCardPayment) {
            return amount * 1.02;       // 2% fee
        }
        else if (payment instanceof UPIPayment) {
            return amount * 1.01;       // 1% fee
        }
        else if (payment instanceof NetBankingPayment) {
            return amount * 1.015;      // 1.5% fee
        }

        return amount;
    }
}

public class Main5 {
    public static void main(String[] args) {
```

```
Scanner sc = new Scanner(System.in);

int N = sc.nextInt();

PaymentProcessor processor = new OnlinePaymentProcessor();

for (int i = 0; i < N; i++) {

    int paymentType = sc.nextInt();
    double amount = sc.nextDouble();

    Payment payment;

    if (paymentType == 1) {
        payment = new CreditCardPayment();
    }
    else if (paymentType == 2) {
        payment = new UPIPayment();
    }
    else {
        payment = new NetBankingPayment();
    }

    double finalAmount =
            processor.processPayment(payment, amount);

    if (paymentType == 1) {
        System.out.printf("CreditCard %.2f%n", finalAmount);
    }
    else if (paymentType == 2) {
```

```
            System.out.printf("UPI %.2f%n", finalAmount);
        }
        else {
            System.out.printf("NetBanking %.2f%n", finalAmount);
        }
    }

    sc.close();
  }
}
```

Input:

3

1 1000

2 2000

3 5000

```
C:\Users\K Bhavya sree>java Main
3
1 1000
2 2000
CreditCard 1020.00
UPI 2020.00
3 5000
```

5.
```
import java.util.*;

public class Solution {

  public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    try {
      int a = sc.nextInt();
      int b = sc.nextInt();
```

```
            System.out.println(a / b);
        }
        catch (InputMismatchException e) {
            System.out.println(e.getClass().getName());
        }
        catch (ArithmeticException e) {
            System.out.println(e);
        }

        sc.close();
    }
}
```

nput (stdin)

```
10
3
```

Expected Output

```
3
```

Infosys®
Navigate your next

COURSE COMPLETION CERTIFICATE

The certificate is awarded to

ABIRAMI T

for successfully completing the course

Computational Problem Solving

on September 18, 2026

Infosys | Springboard

Congratulations! You make us proud!

Satheesha B. Nanjappa
Senior Vice President and Head
Education, Training and Assessment
Infosys Limited

Issued on: Friday, September 18, 2026
To verify, scan the QR code at https://verify.onwingspan.com